

Université Claude Bernard  Lyon 1

Université Claude Bernard Lyon 1
Master Mathématiques Appliquées, Statistique

Projet final de l'unité d'enseignement "Deep Learning"

Étudiants : **SALAZAR Antoine, N'GUESSAN Konan Gervais**

Enseignant : **CHRETIEN Stephane**

Année universitaire 2025–2026

6 mars 2026

Introduction

Dans le cadre du projet final de l'unité d'enseignement « Deep Learning », nous avons mené une étude portant sur la prévision de séries temporelles à l'aide de l'architecture **Temporal Fusion Transformer (TFT)**. Notre démarche s'est initialement appuyée sur les travaux de **Tristan Marcelin (2020)**. L'analyse approfondie et l'adaptation des codes sources associés à son rapport nous ont permis non seulement de nous approprier les modèles étudiés, mais également de reproduire et valider les conclusions de son rapport.

À ce titre, nous tenons à saluer la qualité du travail réalisé par **Tristan Marcelin**, dont l'étude a constitué le socle de notre projet. La clarté et la précision de ses explications ont grandement facilité notre prise en main du sujet et optimisé la réalisation de notre travail.

Table des matières

1	Contexte et Objectifs	3
2	Présentation des données analysées	3
3	Modèle "Long Short-Term Memory" (LSTM)	4
3.1	Description et explication du code	4
3.1.1	Préparation des données	4
3.1.2	Construction du modèle	6
3.2	Entraînement du modèle	7
3.3	Premières prédictions et interprétation	8
3.4	Perspectives d'amélioration du modèle	10
4	Incertitude fréquentiste dans les réseaux de neurones récurrents via des fonctions d'influence par blocs	11
4.1	Introduction	11
4.1.1	Le défi de l'incertitude dans les réseaux récurrents (LSTM)	11
4.1.2	L'apport de la méthode d'Alaa et van der Schaar (ICML 2020)	11
4.1.3	Vers une incertitude native : Le lien avec le TFT	11
4.2	Modifications du code	12
4.3	Explication du code	12
4.4	Évaluation et Visualisation de l'Incertitude	17
4.4.1	Construction du graphique	17
4.4.2	Interprétation des Résultats	18
4.5	Conclusion et Perspectives	19
4.5.1	Bilan de l'approche par Fonctions d'Influence	19
4.5.2	Limites et transition vers le Temporal Fusion Transformer (TFT)	19
5	Temporal Fusion Transformer : TFT	20
5.1	Description : Un modèle hybride de prédiction par quantiles	23
5.1.1	Fonctionnement du code	24
5.2	Prédiction de la Bourse Apple cas Données normales vs bruitées AAPL	39
5.2.1	Réseaux de sélection de variables	42
6	Limites du modèle TFT	44
6.1	Un comportement excessivement conservateur surtout face au chaos	44
6.2	Une perte d'exploitabilité financière liée à l'incertitude	44
6.3	Une forte dépendance à la profondeur de l'historique d'apprentissage	44
7	Conclusion générale et perspective	45

1 Contexte et Objectifs

Marcelin [2020] cherche à quantifier l'incertitude d'un modèle **Long Short Term Theory (LSTM)** sur les marchés financiers. Le **LSTM**, appelé en français réseau de neurones artificiel récurrents à mémoire de court et de long terme, est un modèle Deep Learning, précisément un réseau de neurones récurrents (appelé **RNN**) créé en 1997 par Hochreiter et Schmidhuber mais réellement exploité que depuis une quinzaine d'années. Contrairement aux réseaux de neurones classiques, le **LSTM** a des connexions rétroactives ce qui lui permet de traiter des séquences entières de données.

L'idée derrière ce projet est donc double : d'une part, réaliser des prédictions sur les marchés financiers à l'aide d'architectures séquentielles (le **LSTM** et le **Temporal Fusion Transformer - TFT**) ; d'autre part, être capable de quantifier rigoureusement l'incertitude inhérente à ces prédictions.

Notre objectif sera, dans un premier temps, de comprendre le fonctionnement d'un modèle de base **LSTM** et d'étudier une méthode de quantification de l'incertitude *fréquentiste* développée par Alaa and van der Schaar [2020]. Cette méthode repose sur l'approximation de la matrice Hessienne et les fonctions d'influence.

Nous disposons pour cela de modèles "prototypes" implémentés en Python, que nous testerons sur nos propres données financières. Sur ces modèles, nous réaliserons des analyses poussées, en introduisant notamment un bruit artificiel hétéroscédastique dans nos données afin d'observer comment le modèle résiste à la perturbation et adapte ses intervalles de confiance.

Dans un second temps, nous déploierons sur ce même jeu de données le modèle **TFT**. Cette architecture de pointe, utilisée par Marcelin [2020], intègre nativement la quantification de l'incertitude grâce à une fonction de perte spécifique (la régression par quantiles). Cela nous permettra de comparer les performances prédictives et la gestion de l'incertitude des deux approches. Notre cadre d'étude restera celui des marchés financiers, assurant ainsi la continuité avec le précédent rapport.

2 Présentation des données analysées

Dans l'objectif de mieux comprendre le fonctionnement des réseaux de neurones évoqués plus haut, nous allons entraîner les différents algorithmes avec un jeu de données spécifique. Il a été choisi ici d'importer le package **yfinance** qui est un outil permettant de réaliser de l'analyse de données financières avec Python. Cette bibliothèque sert de "pont" entre Python et le site Yahoo Finance. Elle permet de récupérer des années de données historiques et les affiche sous la forme de dataframe **pandas**.

Dans la suite de ce rapport, nous allons concentrer notre analyse sur le cours d'ouverture de l'action de l'entreprise "**Apple**", entreprise multinationale américaine qui crée et commercialise des produits électroniques grand public, des ordinateurs personnels et des logiciels. Le choix de cette entreprise repose sur la volonté de récolter le plus grand nombre de données possibles pour notre analyse, ce qui est faisable dans ce cas puisque les données financières de l'entreprise Apple sont facilement collectables.

Price	Close	High	Low	Open	Volume
Ticker	AAPL	AAPL	AAPL	AAPL	AAPL
Date					
2011-01-03	9.874907	9.895582	9.733183	9.757153	445138400
2011-01-04	9.926446	9.962701	9.832362	9.960903	309080800
2011-01-05	10.007645	10.017832	9.872811	9.874310	255519600
2011-01-06	9.999554	10.045097	9.974684	10.029217	300428800
2011-01-07	10.071161	10.078053	9.944717	10.007340	311931200
...	...	...	...	...	...
2018-12-24	34.870010	35.990943	34.813013	35.183489	148676800
2018-12-26	37.325603	37.339852	34.843880	35.219108	234330000
2018-12-27	37.083382	37.230625	35.639472	37.009762	212468400
2018-12-28	37.102375	37.646219	36.703401	37.403983	169165600
2018-12-31	37.460976	37.845701	37.161742	37.648588	140014000

2012 rows × 5 columns

FIGURE 1 – Données quotidiennes concernant le cours de l'action de l'entreprise "Apple"

Nous étudierons ici l'évolution quotidienne du cours de l'action à l'ouverture entre le 01/01/2011 et le 01/01/2019. Les données ne sont récoltées que sur les jours ouvrés. Nous disposons donc de 2012 observations sur le cours de l'action Apple à l'ouverture. L'objectif derrière la focalisation sur le cours à l'ouverture est de rester fidèle aux études précédentes faites en ce sens.

3 Modèle "Long Short-Term Memory" (LSTM)

Nous allons dans cette section présenter l'architecture d'un réseau **LSTM**. Cette présentation s'appuiera sur l'analyse du code Python construisant ce modèle, fourni en annexe de ce projet. Le modèle présenté ici fait office de prototype et servira de base de comparaison (*baseline*) pour le modèle **TFT** que nous détaillerons par la suite.

3.1 Description et explication du code

3.1.1 Préparation des données

Après l'extraction des données historiques de l'entreprise Apple, une première fonction `sliding_windows` est appliquée. Son objectif est de séquencer les données pour adapter le format d'entrée aux exigences du modèle LSTM. Elle prend deux paramètres : **les données brutes** et `seq_length`. Ce dernier définit la fenêtre d'observation, c'est-à-dire la profondeur de l'historique que le modèle utilise pour effectuer sa prédiction. Dans notre

implémentation, ce paramètre est fixé à 4 : pour prédire le cours d'ouverture au jour i , le modèle s'appuie sur les 4 jours précédents.

Ce fenêtrage génère deux ensembles : un tenseur X représentant les séquences historiques et un tenseur Y contenant les valeurs cibles.

```
1 def sliding_windows(data, seq_length):
2     x = []
3     y = []
4
5     for i in range(len(data)-seq_length-1):
6         _x = data[i:(i+seq_length)]
7         _y = data[i+seq_length]
8         x.append(_x)
9         y.append(_y)
10
11     return np.array(x), np.array(y)
```

Listing 1 – Création des séquences temporelles par fenêtre glissante

Afin d'optimiser l'apprentissage, ces données sont ensuite normalisées sur un intervalle allant de 0 à 1 via la fonction `MinMaxScaler`. Cette étape de mise à l'échelle est cruciale : elle stabilise et accélère la convergence lors de la descente de gradient.

Le jeu de données est ensuite scindé : 70 % des observations sont allouées à l'ensemble d'entraînement et 30 % constituent l'ensemble de test. Enfin, ces ensembles sont convertis en tenseurs compatibles avec PyTorch.

```
1 sc = MinMaxScaler()
2 training_data = sc.fit_transform(train_set)
3
4 seq_length = 4
5 x, y = sliding_windows(training_data, seq_length)
6
7 # Decoupage Train/Test (70% / 30%)
8 train_size = int(len(y) * 0.70)
9 test_size = len(y) - train_size
10
11 # Conversion en tenseurs PyTorch
12 trainX = Variable(torch.Tensor(np.array(x[0:train_size])))
13 trainY = Variable(torch.Tensor(np.array(y[0:train_size])))
```

Listing 2 – Normalisation et découpage des données

3.1.2 Construction du modèle

```

1 class LSTM(nn.Module):
2     def __init__(self, num_classes, input_size, hidden_size, num_layers)
3         :
4         super(LSTM, self).__init__()
5
6         self.num_classes = num_classes
7         self.num_layers = num_layers
8         self.input_size = input_size
9         self.hidden_size = hidden_size
10        self.seq_length = seq_length
11
12        # Couche recurrente LSTM
13        self.lstm = nn.LSTM(input_size=input_size, hidden_size=hidden_
14                               size,
15                               num_layers=num_layers, batch_first=True)
16
17        # Couche lineaire (Fully Connected)
18        self.fc = nn.Linear(hidden_size, num_classes)
19
20    def forward(self, x):
21        # Initialisation des etats caches
22        h_0 = Variable(torch.zeros(self.num_layers, x.size(0), self.
23                                    hidden_size))
24        c_0 = Variable(torch.zeros(self.num_layers, x.size(0), self.
25                                    hidden_size))
26
27        # Propagation de l'entree dans le LSTM
28        ula, (h_out, _) = self.lstm(x, (h_0, c_0))
29
30        # Aplatissement et prediction finale
31        h_out = h_out.view(-1, self.hidden_size)
32        out = self.fc(h_out)
33
34        return out

```

Listing 3 – Architecture de la classe LSTM

Dans notre implémentation, le modèle LSTM est défini via une classe héritant de `nn.Module`, la classe de base de PyTorch pour tous les réseaux de neurones. Cette classe `LSTM`, est composée de deux fonctions principales :

- **Initialisation** (`__init__`) : Cette fonction prépare les composants du modèle. L'appel `super(LSTM, self).__init__()` y est obligatoire. On y retrouve :
 - `self.lstm` : C'est le cœur du modèle. Il définit la taille des données (`input_size`), la mémoire interne (`hidden_size`) et le nombre de couches (`num_layers`). L'option `batch_first=True` organise les données en (`batch`, `sequence`, `feature`).
 - `self.fc` : Une couche linéaire qui transforme la sortie du **LSTM** pour correspondre au nombre de classes ou à la valeur prédite.
- **Propagation** (`forward`) : Cette fonction définit comment les données circulent dans le réseau lors d'une prédiction. Le flux suit ces étapes :

- **Initialisation des états** (h_0 et c_0) : Le **LSTM** nécessite deux vecteurs de mémoire initiaux, l'état caché (h_0) et l'état de la cellule (c_0), souvent initialisés à zéro.
- **Propagation** (`self.lstm`) : Le modèle traite la séquence $\mathbf{x}$. `h_out` contient le dernier état caché, qui est essentiellement le "résumé" de toute la séquence de 4 jours que le modèle vient de lire.
- **Redimensionnement** (`view`) : Le code utilise `view(-1, self.hidden_size)` pour "aplatir" la sortie afin qu'elle puisse entrer dans la couche finale.
- **Sortie finale** (`textttself.fc`) : La mémoire est transformée en une prédiction concrète (le prix de l'action).

3.2 Entraînement du modèle

La phase d'apprentissage consiste à ajuster les paramètres du modèle sur l'ensemble d'entraînement. Deux éléments mathématiques et algorithmiques encadrent ce processus :

- **Fonction de perte** : Pour mesurer la performance du réseau **LSTM** lors de la phase d'entraînement, nous utilisons la fonction de perte **MSE** (*Mean Squared Error*), implémentée via la ligne de code `criterion = torch.nn.MSELoss()`. Elle est définie par l'équation suivante :

$$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2 \quad (1)$$

- n : Représente le nombre total d'observations (ou d'échantillons) présents dans le jeu de données (ici $n = 2012$).
- $\sum_{i=1}^n$: Le symbole de sommation indique que l'on additionne les résultats obtenus pour chaque échantillon i allant de 1 à n .
- Y_i : La valeur réelle attendue. Dans le script, cela correspond à la variable `trainY`, soit le prix réel de l'action Apple.
- $\hat{Y}_i$: La valeur prédite par le modèle **LSTM**. Dans le code PyTorch, cela correspond à la variable `outputs`.

L'erreur entre la réalité (Y_i) et la prédiction ($\hat{Y}_i$) est élevée au carré, $(Y_i - \hat{Y}_i)^2$, pour deux raisons méthodologiques :

- **Positivité de l'erreur** : Cela garantit que la valeur de l'erreur est toujours positive, évitant ainsi que des erreurs négatives ne viennent annuler des erreurs positives lors du calcul de la somme.
- **Pénalisation des écarts** : Cette fonction pénalise beaucoup plus lourdement les erreurs importantes que les erreurs mineures.

- `optimizer = torch.optim.Adam` : On utilise l'optimiseur **Adam**, un algorithme qui ajuste les poids internes du modèle pour réduire la perte. On lui passe `learning_rate` (le taux d'apprentissage), qui définit la taille des pas effectués lors de l'ajustement des poids. Mathématiquement, la pénalisation des écarts évoquée plus haut force l'optimiseur **Adam** à corriger en priorité les prédictions les plus éloignées de la réalité pour accroître la précision globale du modèle.

Le cycle d'entraînement est répété sur 1000 époques. Pour chaque itération, le modèle effectue une propagation avant, calcule l'erreur, réinitialise ses gradients, applique la rétropropagation (`backward`) et met à jour ses poids (`step`).

- **Propagation Avant** (`outputs = lstm(trainX)`) : Le modèle prend les séquences d'entrée et tente de prédire les prix correspondants.
- **Remise à zéro** (`optimizer.zero_grad()`) : On efface les calculs d'erreurs précédents pour ne pas les accumuler lors de cette nouvelle étape.
- **Calcul de l'erreur** (`loss = criterion(outputs, trainY)`) : On compare la prédiction (`outputs`) avec la réalité (`trainY`).
- **Rétropropagation** (`loss.backward()`) : Étape de PyTorch où le code calcule l'influence de chaque paramètre du modèle sur l'erreur totale. Cela correspond au calcul du gradient.
- **Mise à jour** (`optimizer.step()`) : L'optimiseur modifie les poids du modèle dans la direction qui devrait réduire l'erreur lors du prochain passage.

```

1 lstm = LSTM(num_classes, input_size, hidden_size, num_layers)
2
3 criterion = torch.nn.MSELoss()      # Erreur Quadratique Moyenne
4 optimizer = torch.optim.Adam(lstm.parameters(), lr=learning_rate)
5
6 # Entraînement du modele
7 for epoch in range(num_epochs):
8     outputs = lstm(trainX)
9     optimizer.zero_grad()
10
11     # Calcul de la perte
12     loss = criterion(outputs, trainY)
13
14     # Retropropagation et mise a jour des poids
15     loss.backward()
16     optimizer.step()

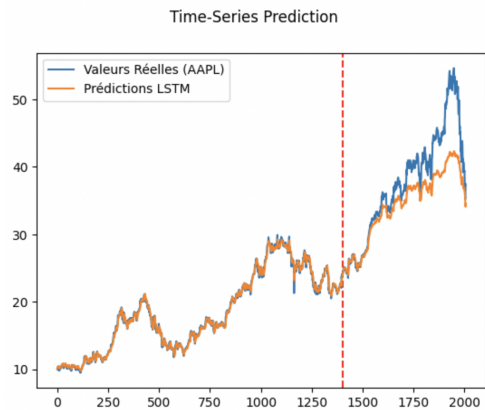
```

Listing 4 – Boucle d'entraînement du modèle LSTM

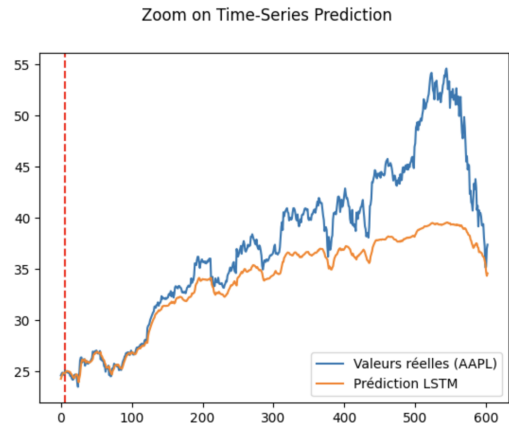
3.3 Premières prédictions et interprétation

L'évaluation de ce prototype s'est effectuée sur l'ensemble des données (2012 observations). Avant l'analyse visuelle, une transformation inverse (`sc.inverse_transform`)

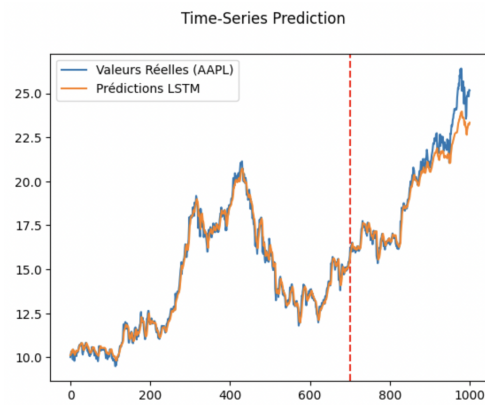
est appliquée aux prédictions pour ramener les valeurs de l'intervalle $[0, 1]$ à leur échelle monétaire d'origine (en dollars), permettant ainsi leur interprétation.



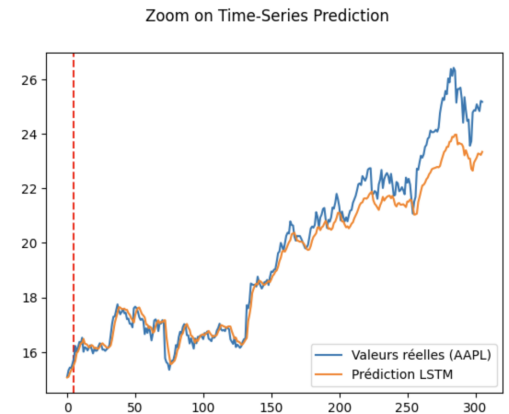
(a) Prédictions sur la période 2011-2019



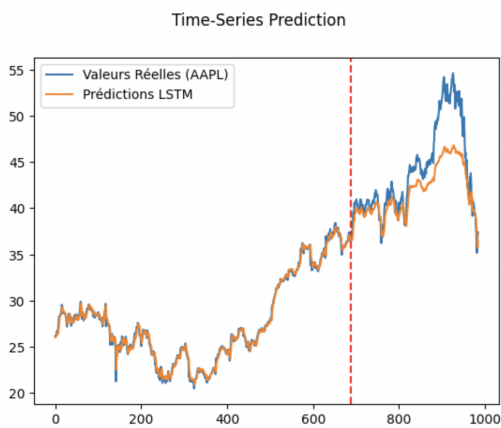
(b) Zoom sur les prédictions pour l'ensemble de test sur la période 2011-2019



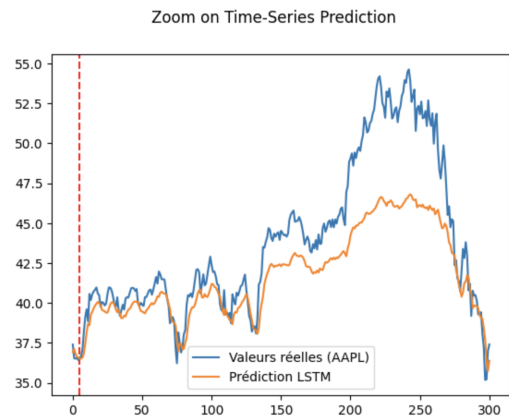
(c) Prédictions sur la période 2011-2015



(d) Zoom sur les prédictions pour l'ensemble de test sur la période 2011-2015



(e) Prédictions sur la période 2015-2019



(f) Zoom sur les prédictions pour l'ensemble de test sur la période 2015-2019

FIGURE 2 – Prédictions du cours réel de l'action Apple par le modèle LSTM entre 2011 et 2019.

L'observation graphique des résultats met en évidence un excellent ajustement du modèle sur les données d'entraînement. Sur l'ensemble de test, le modèle **LSTM** parvient à identifier la tendance globale de l'action. Néanmoins, on observe un décalage et des erreurs significatives lors des périodes de forte volatilité boursière. Ces écarts de prédiction confirment que si le **LSTM** est capable d'apprendre des tendances lissées, il atteint ses limites face à de soudaines variations extrêmes.

Une analyse plus fine sur un sous-ensemble (2011-2015), caractérisé par une variance plus modérée, démontre une meilleure convergence des prédictions, confirmant la sensibilité de cette architecture de base aux pics de volatilité.

3.4 Perspectives d'amélioration du modèle

Afin de pallier la réactivité tardive du modèle face aux fluctuations abruptes, nous proposons une optimisation des hyperparamètres, sans pour autant modifier fondamentalement l'architecture :

1. **Augmentation de la dimension cachée** : Passer `hidden_size` de 2 à 32, pour accroître la capacité du réseau à mémoriser des motifs complexes.
2. **Élargissement de la fenêtre glissante** : Augmenter `seq_length` de 4 à 20 jours (un mois de bourse) pour intégrer un contexte historique plus large.
3. **Augmentation du nombre de couches** : Faire passer `num_layers` de 1 à 2.

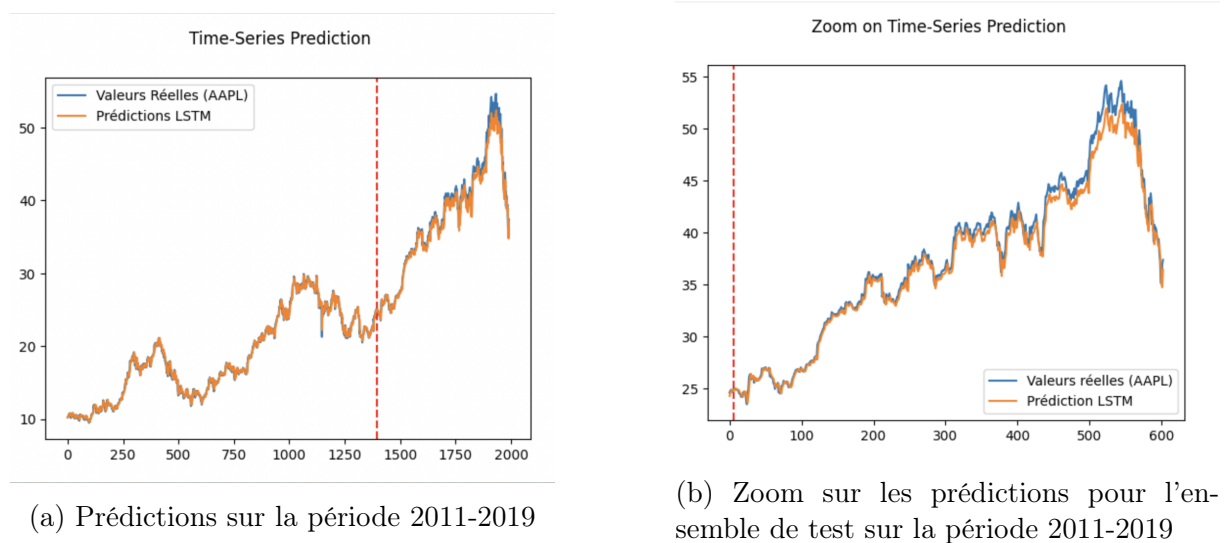


FIGURE 3 – Prédictions du cours réel de l'action Apple par le modèle LSTM entre 2011 et 2019 après modification des paramètres.

La modification des paramètres donne des résultats presque optimaux puisque nous pouvons observer que les écarts entre les deux courbes sont encore plus faibles.

Ce modèle **LSTM** permet donc de réaliser des prédictions précises pour un tel jeu de données. Cependant subsiste la problématique de l'incertitude liée aux potentielles erreurs de prédiction. Nous allons revenir plus en détail sur ce point dans la section suivante.

4 Incertitude fréquentiste dans les réseaux de neurones récurrents via des fonctions d'influence par blocs

4.1 Introduction

Dans le domaine de la prévision de séries temporelles, fournir une simple estimation ponctuelle (une valeur unique) est souvent insuffisant, particulièrement dans des secteurs critiques comme la finance ou la santé. Il est crucial de pouvoir quantifier l'incertitude associée à une prédiction. Là était l'objectif du rapport de Marcelin.

4.1.1 Le défi de l'incertitude dans les réseaux récurrents (LSTM)

Les **Réseaux de Neurones Récurrents (RNN)**, et plus particulièrement les architectures **LSTM (Long Short-Term Memory)**, se sont imposés comme des standards pour capturer les dépendances temporelles complexes. Cependant, ces modèles sont fondamentalement déterministes : ils ne fournissent pas nativement d'intervalles de confiance. Pour pallier ce manque, les approches traditionnelles se tournent généralement vers les réseaux de neurones bayésiens ou le *Monte Carlo Dropout*. Bien que performantes, ces méthodes bayésiennes sont souvent très coûteuses en temps de calcul et nécessitent de modifier l'architecture ou le processus d'entraînement du modèle.

4.1.2 L'apport de la méthode d'Alaa et van der Schaar (ICML 2020)

C'est ici que réside l'intérêt majeur des travaux de Alaa and van der Schaar [2020]. Dans leur article, ils proposent une approche *fréquentiste* et *post-hoc* pour quantifier l'incertitude dans les **RNN**.

Leur idée consiste à utiliser les **Fonctions d'Influence** (Influence Functions). Plutôt que de réentraîner le modèle des milliers de fois (comme le ferait une méthode de *Leave-One-Out*), cette méthode utilise la matrice Hessienne pour approximer mathématiquement l'impact de chaque donnée d'entraînement sur les poids finaux du réseau. L'avantage est double :

- **Efficacité** : L'incertitude est calculée une fois le modèle entraîné, sans en altérer l'architecture.
- **Rigueur fréquentiste** : Elle permet de générer des intervalles de confiance séquentiels fiables face à un bruit hétéroscédastique (évoluant dans le temps).

4.1.3 Vers une incertitude native : Le lien avec le TFT

Si l'approche d'Alaa and van der Schaar [2020] offre une solution élégante pour doter a posteriori un LSTM d'intervalles de confiance, la recherche récente s'est orientée vers des architectures intégrant l'incertitude de manière native.

Le **Temporal Fusion Transformer (TFT)** représente l'évolution naturelle de cette problématique. Là où les **LSTM** traitent l'information de manière séquentielle, le **TFT** utilise des mécanismes d'attention pour identifier les dépendances à long terme. Surtout, au lieu d'estimer l'incertitude *post-hoc*, le **TFT** est généralement entraîné avec une fonction de perte spécifique (la régression quantile - *Quantile Loss*).

Il apprend ainsi à prédire simultanément plusieurs quantiles (par exemple le 10e, le 50e

et le 90e percentile), fournissant un intervalle de confiance *by design*.

Le code étudié dans ce rapport constitue donc un jalon conceptuel essentiel : il illustre comment mathématiser rigoureusement l'incertitude sur une architecture récurrente classique, posant ainsi les bases d'une réflexion qui mènera aux modèles prédictifs modernes multi-horizons comme le **TFT**.

4.2 Modifications du code

Le code fournit ne pouvait pas être exécutable tel quel. Cela est dû à l'ancienneté du code et à la version de **Pytorch** utilisée qui n'est plus compatible avec certaines versions actuelles d'autres bibliothèques. Un problème majeur que nous avons rencontré concerne l'incompatibilité entre la version 1.4.0 de Pytorch (datant de 2020) et les versions actuelles de Numpy. Pour cela, il a donc fallu installer des mises à jour pour permettre à notre logiciel de pouvoir faire fonctionner toutes les bibliothèques ensemble. La commande `!pip install -upgrade torch torchvision numpy` met à jour les différentes versions des bibliothèques pour les rendre compatibles entre elles.

Egalement, le code fournit faisait appel à d'autres fonctions décrites dans des modules `py`. Il fallait donc importer ces modules `py` pour pouvoir exécuter le code correctement. Nous avons choisi pour faciliter la lecture du code par le prochain lecteur de réécrire les fonctions nécessaires directement dans le premier code. Cela augmente certes la taille de celui-ci mais permet à celui qui ne disposera pas de tous les éléments de pouvoir exécuter le code avec un seul fichier. Nous décrirons toutes ces fonctions dans la partie suivante.

4.3 Explication du code

Préparation des données

Ce deuxième code est similaire au premier sur les **LSTM** dans le sens où la préparation des données se fait de manière identique. Nous avons donc une première partie où nous chargeons le jeu de données du cours de l'action "Apple", puis une deuxième partie dans laquelle on définit la fonction `sliding_windows` qui définit la mémoire de notre modèle dans son apprentissage. Là encore, par défaut la taille de cette mémoire est fixée à 4, ce qui veut dire que le modèle fonde sa prédiction pour un jour i en se fondant sur les 4 jours précédents celui-ci. Nous avons vu précédemment que l'augmentation de cette fenêtre permet un meilleur apprentissage et de meilleures prédictions.

Après avoir défini l'ajustement des paramètres, on définit deux fonctions, `autoregressive` et `create_autoregressive_data`.

Pour évaluer l'incertitude fréquentiste, nous générons des données autorégressives où chaque échantillon est une séquence de longueur T . Le processus est défini par :

$$X_t \sim \mathcal{N}(\mu_X, \sigma_X^2) \quad (2)$$

$$y_t = \left(\sum_{i=0}^t X_i \alpha^{t-i} \right) + \epsilon_t \quad (3)$$

$$\epsilon_t \sim \mathcal{N}(0, \sigma_t^2) \quad (4)$$

où σ_t augmente avec t , créant ainsi une incertitude temporelle dépendante.

Génération de Données Synthétiques

Pour évaluer la capacité de notre modèle à quantifier l'incertitude, nous générons des séries temporelles synthétiques. Ce processus repose sur deux fonctions principales implémentant un modèle autorégressif avec un bruit hétéroscédastique (dont la variance évolue dans le temps).

Le modèle autorégressif déterministe

La fonction `autoregressive` calcule la composante déterministe de la série. Pour une séquence d'entrée X , la valeur à l'instant k , notée y_k , dépend de l'historique des valeurs de X jusqu'à l'instant k , pondérées par un vecteur mémoire w . Mathématiquement, cela correspond à une convolution temporelle discrète :

$$y_k = \sum_{i=0}^k X_i \cdot w_{k-i} \quad (5)$$

L'implémentation en Python utilise la bibliothèque `numpy` pour optimiser ce calcul via une somme pondérée sur les poids inversés grâce à la fonction `np.flip` :

```
1 def autoregressive(X, w):
2     return np.array([
3         np.sum(X[0:k+1] * np.flip(w[0:k+1]).reshape(-1, 1))
4         for k in range(len(X))
5     ])
```

Listing 5 – Fonction de calcul de la composante autorégressive

Ajout du bruit hétéroscédastique et création du dataset

La fonction `create_autoregressive_data` orchestre la création du jeu de données final. Le processus suit trois étapes fondamentales :

1. **Génération des entrées** : Les séquences d'entrée sont échantillonnées de manière aléatoire selon une distribution normale :

$$X_k \sim \mathcal{N}(\mu_X, \sigma_X^2) \quad (6)$$

2. **Décroissance de la mémoire** : Le vecteur de poids w simule une mémoire qui s'estompe avec le temps selon une décroissance exponentielle :

$$w_k = \rho^k, \quad \text{où } \rho \in [0, 1] \quad (7)$$

Dans notre code, ρ correspond au paramètre `memory_factor`.

3. **Ajout du bruit (hétéroscédasticité)** : Le modèle cible Y intègre la composante autorégressive calculée précédemment et y ajoute un bruit ϵ_k . La variance de ce bruit augmente dans le temps, définie par un profil de bruit σ_k (paramètre `noise_profile`) :

$$Y_k = y_k + \epsilon_k \quad (8)$$

$$\epsilon_k \sim \mathcal{N}(0, \sigma_k^2) \quad (9)$$

Cette variance temporelle croissante de ϵ_k est cruciale dans notre expérimentation : elle force le réseau de neurones à modéliser une incertitude de plus en plus grande à mesure que la séquence avance dans le temps.

```

1 def create_autoregressive_data(n_samples=100, seq_len=6, n_features=1,
2                               X_m=1, X_v=2,
3                               noise_profile=[0, 0.2, 0.4, 0.6, 0.8, 1],
4                               memory_factor=0.9, mode="time-dependent")
5                               :
6     # 1. Generation des entrees X (distribution normale)
7     X = [np.random.normal(X_m, X_v, (seq_len, n_features))
8           for k in range(n_samples)]
9
10    # 2. Vecteur memoire (decroissance exponentielle)
11    w = np.array([memory_factor**k for k in range(seq_len)])
12
13    # 3. Calcul des cibles Y avec l'ajout progressif du bruit temporel
14    if mode == "time-dependent":
15        Y = [
16            (autoregressive(X[k], w).reshape(seq_len, n_features)
17             + np.random.normal(0, noise_profile).reshape(-1, n_features))
18            .reshape(seq_len,)
19            for k in range(n_samples)
20        ]
21
22    return X, Y

```

Listing 6 – Génération complète du jeu de données avec bruit temporel

Modélisation par Réseau de Neurones Récurrents (RNN)

Avant d'estimer l'incertitude, un modèle prédictif de base doit être entraîné. Le code implémente une architecture flexible (**RNN standard**, **LSTM** ou **GRU**) via la bibliothèque PyTorch.

Gestion des séquences de taille variable : Padding et Masquage

Dans la pratique, les séries temporelles peuvent avoir des longueurs inégales. Pour les traiter en lots (batches), la fonction `padd_arrays` uniformise leur taille à une longueur maximale définie (`MAX_STEPS`) en les complétant avec des zéros.

Pour éviter que le réseau n'apprenne sur ces zéros artificiels, un masque binaire M est généré de manière conjointe :

$$M_{i,t} = \begin{cases} 1 & \text{si la donnée à la séquence } i \text{ et au temps } t \text{ est réelle} \\ 0 & \text{si c'est un zéro de remplissage (padding)} \end{cases} \quad (10)$$

Fonction de perte masquée (Masked Loss)

L'entraînement du modèle s'effectue en minimisant l'Erreur Quadratique Moyenne (MSE). Cependant, cette erreur doit être pondérée par le masque M pour que seules les vraies valeurs contribuent au gradient.

Pour une prédiction $\hat{y}$ et une cible y , l'erreur quadratique individuelle masquée est :

$$\mathcal{L}_{i,t} = M_{i,t} \cdot (\hat{y}_{i,t} - y_{i,t})^2 \quad (11)$$

La perte globale sur un lot (batch) est la somme des erreurs individuelles normalisée par le nombre total de vraies valeurs (la somme des éléments du masque) :

$$\mathcal{L}_{batch} = \frac{\sum_i \sum_t \mathcal{L}_{i,t}}{\sum_i \sum_t M_{i,t}} \quad (12)$$

L'implémentation PyTorch de cette fonction de perte vectorisée garantit un calcul rapide et différentiable pour la rétropropagation :

```

1 def model_loss(output, target, masks):
2     # Calcul de l'erreur au carré, annulée la où le masque vaut 0
3     single_loss = masks * (output - target)**2
4
5     # Somme des erreurs normalisée par la somme totale du masque
6     loss = torch.sum(
7         torch.sum(single_loss, axis=1) / torch.sum(torch.sum(masks, axis
8             =1))
9     )
10    return loss
11
12 def single_losses(model):
13     # Retourne les pertes individuelles non agrégées
14     # Utile plus tard pour le calcul des Fonctions d'Influence
15     return model.masks * (model(model.X).view(-1, model.MAX_STEPS) -
        model.y)**2

```

Listing 7 – Calcul de la perte masquée (Masked Loss)

Entraînement du modèle

L'architecture, encapsulée dans la classe RNN, procède à l'entraînement classique par descente de gradient via l'optimiseur Adam. Il est crucial de noter que la méthode `fit` stocke les tenseurs originaux (`self.X`, `self.y`, `self.masks`) après l'entraînement. Cette étape est indispensable pour la suite, car l'estimation de l'incertitude fréquentiste nécessitera de recalculer les gradients sur ces mêmes données d'entraînement.

Estimation de l'Incertainitude Fréquentiste par Fonctions d'Influence

L'apport principal de la méthode (Alaa & van der Schaar, ICML 2020) réside dans le calcul post-hoc de l'incertitude. Plutôt que d'adopter une approche bayésienne coûteuse ou de réentraîner le modèle de multiples fois en retirant des données (Leave-One-Out), le code utilise des *Influence Functions* (Fonctions d'Influence) pour approximer mathématiquement l'impact de chaque séquence d'entraînement sur les paramètres finaux du modèle.

Théorie Mathématique des Fonctions d'Influence

Soit $\hat{\theta}$ les paramètres optimaux du modèle obtenus après l'entraînement. L'objectif est d'estimer comment $\hat{\theta}$ changerait si une donnée d'entraînement z_i (une séquence temporelle) était retirée ou perturbée.

La variation des paramètres, notée $\mathcal{I}(z_i)$, est donnée par l'approximation de Taylor au premier ordre :

$$\mathcal{I}(z_i) = \left. \frac{d\hat{\theta}}{d\epsilon} \right|_{\epsilon=0} = -H_{\hat{\theta}}^{-1} \nabla_{\theta} \mathcal{L}(z_i, \hat{\theta}) \quad (13)$$

où :

- $H_{\hat{\theta}} = \frac{1}{N} \sum_{j=1}^N \nabla_{\theta}^2 \mathcal{L}(z_j, \hat{\theta})$ est la matrice Hessienne (les dérivées secondes de la perte sur tout l'ensemble d'entraînement).
- $\nabla_{\theta} \mathcal{L}(z_i, \hat{\theta})$ est le gradient de la fonction de perte pour le point d'entraînement z_i .

Implémentation du Calcul de la Hessienne Inverse

L'implémentation de cette formule nécessite l'inversion de la matrice Hessienne. Le code propose deux méthodes via la fonction `influence_function` :

1. **Mode Exact** (`exact_influence`) : Calcule la Hessienne complète et son inverse exacte à l'aide de `torch.autograd.grad` et `torch.inverse`. Cela nécessite l'ajout d'un terme d'amortissement (*damping*, λI) pour garantir que la matrice soit inversible : $H_{inv} = (H + \lambda I)^{-1}$.
2. **Mode Stochastique** (`influence_stochastic_estimation`) : Pour les très grands modèles, l'inversion de H est impossible. Le code utilise une approximation par série entière (développement de Lissa) basée sur des produits Matrice-Vecteur (HVP - *Hessian Vector Product*).

```

1 def exact_influence(model, train_index, damp=0, W=None, order=1):
2     # 1. Recuperation des parametres du modele
3     params_ = [param for param in model.parameters()]
4     num_par = stack_torch_tensors(params_).shape[0]
5
6     # 2. Calcul de la Hessienne inverse avec amortissement (damping)
7     Hinv = torch.inverse(exact_hessian(model) + damp * torch.eye(num_par)
8                           ))
9
10    # 3. Calcul des gradients pour chaque point d'entraînement
11    losses = [torch.sum(model.sequence_loss()[train_index[k], :])
12              for k in range(len(train_index))]
13    grads = [stack_torch_tensors(
14              torch.autograd.grad(losses[k], model.parameters(), create_
15                                  graph=True)
16              ) for k in range(len(losses))]
17
18    # 4. Multiplication : -H^{-1} * grad
19    n_factor = torch.sum(model.masks)
20    IFs_ = [-1 * torch.mm(Hinv, grads[k].reshape((grads[k].shape[0], 1))
21              ) / n_factor
22            for k in range(len(grads))]
23
24    return IFs_

```

Listing 8 – Calcul de l'Influence Exacte (Extrait)

Perturbation du Modèle et Intervalles de Confiance

Une fois les vecteurs d'influence $\mathcal{I}(z_i)$ calculés pour chaque point i , on peut générer des "modèles virtuels" en perturbant les poids du modèle de base $\hat{\theta}$:

$$\theta_{perturbé}^{(i)} = \hat{\theta} - \mathcal{I}(z_i) \quad (14)$$

La fonction `perturb_model_` clone le modèle PyTorch et lui assigne ces nouveaux paramètres modifiés, simulant ainsi un modèle qui n'aurait jamais vu la donnée i .

Enfin, la classe `RNN_uncertainty_wrapper` rassemble le tout pour générer l'intervalle de confiance. Lors de l'appel à la fonction `predict` sur une nouvelle séquence de test X_{test} :

1. Elle fait prédire chaque modèle perturbé pour obtenir une distribution empirique de prédictions $\hat{y}_{virtuels}$.
2. Elle calcule les résidus empiriques sur l'ensemble d'entraînement (erreur entre les prédictions perturbées et les vraies valeurs, ou Leave-One-Batch-Out residuals).
3. Elle utilise la fonction `np.quantile` pour extraire les bornes inférieures et supérieures de ces prédictions, ajoutant les résidus pour garantir une probabilité de couverture cible (par exemple 90% ou 95%).

```

1 def predict(self, X_test, coverage=0.95):
2     variable_preds = []
3     num_sequences = np.array(X_test).shape[0]
4
5     # Generation des predictions par les modeles perturbés
6     for k in range(len(self.IF)):
7         perturbed_models = perturb_model_(self.model, self.IF[k])
8         variable_preds.append(perturbed_models.predict(X_test))
9         del perturbed_models
10
11     variable_preds = np.array(variable_preds)
12
13     # Calcul des bornes de l'intervalle avec les quantiles
14     alpha_inf = (1 - coverage) / 2
15     alpha_sup = 1 - alpha_inf
16
17     y_u_approx = np.quantile(variable_preds + residuals, alpha_sup, axis
18                             =0)
19     y_l_approx = np.quantile(variable_preds - residuals, alpha_inf, axis
20                             =0)
21
22     y_pred = self.model.predict(X_test)
23
24     return y_pred, y_l_approx, y_u_approx

```

Listing 9 – Calcul des quantiles pour les Intervalles de Confiance (Wrapper)

4.4 Évaluation et Visualisation de l'Incertitude

Une fois les intervalles de confiance calculés pour les nouvelles données de test, il est essentiel de vérifier visuellement si le modèle a correctement capturé la structure du bruit temporel (l'hétéroscédasticité) que nous avons introduite lors de la génération des données.

4.4.1 Construction du graphique

Pour la visualisation, nous utilisons la bibliothèque `matplotlib.pyplot`. La fonction `fill_between` est particulièrement utile pour représenter l'intervalle de confiance continu (la zone d'incertitude) autour de la prédiction moyenne.

Les trois éléments extraits par notre wrapper (la prédiction moyenne $\hat{y}$, la borne inférieure $\hat{y}_{inf}$ et la borne supérieure $\hat{y}_{sup}$) sont superposés avec les véritables valeurs du jeu de test :

```

1 # Zone rouge transparente pour l'intervalle de confiance (shaded area)
2 plt.fill_between(range(len(Y_test[0])), Y_predicted[1][0], Y_predicted
   [2][0],
3                   color="r", alpha=0.25)
4
5 # Lignes pointillées pour les bornes inferieure et superieure
6 plt.plot(Y_predicted[1][0], linestyle="--", color="r")
7 plt.plot(Y_predicted[2][0], linestyle="--", color="r")
8
9 # Ligne rouge epaisse pour la prediction moyenne du RNN
10 plt.plot(Y_predicted[0][0], linestyle="--", linewidth=3, color="r")
11
12 # Points noirs pour les vraies valeurs cibles (ground truth)
13 plt.scatter(range(len(Y_test[0])), Y_test[0], color="black")
14
15 plt.xlabel("Time step", fontsize=12)
16 plt.ylabel("Prediction", fontsize=12)
17 plt.show()

```

Listing 10 – Visualisation des prédictions et des intervalles de confiance

4.4.2 Interprétation des Résultats

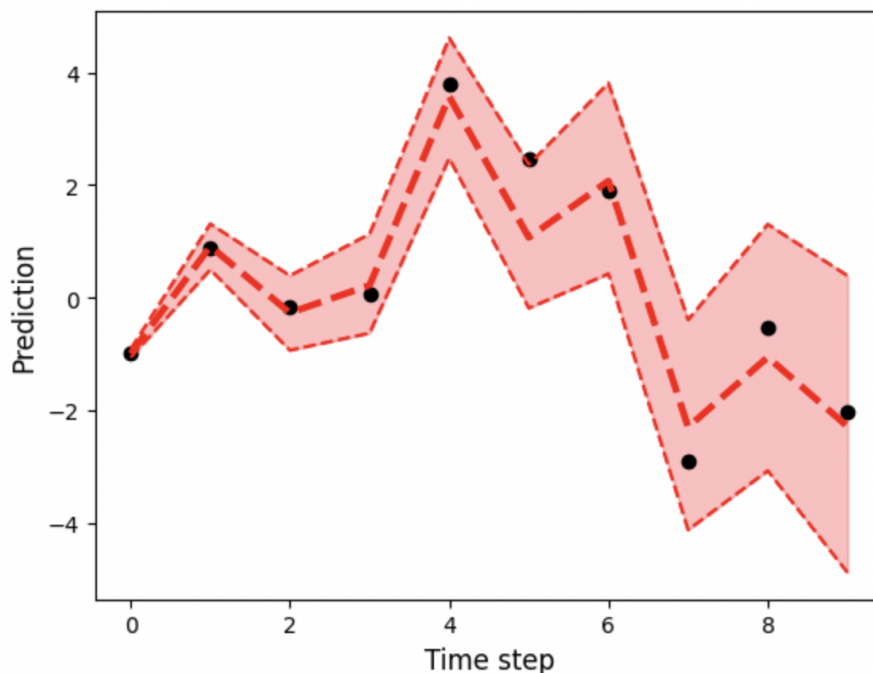


FIGURE 4 – Évolution de la prédiction et de l'intervalle de confiance au cours du temps.

Sur la Figure 4, les points noirs représentent les valeurs réelles de la série temporelle de test, tandis que la courbe rouge en pointillés illustre la prédiction de notre modèle RNN. La zone ombrée en rouge représente l'intervalle de confiance séquentiel estimé via les fonctions d'influence.

Conformément à la conception de nos données synthétiques (où la variance du bruit augmente à chaque pas de temps), nous constatons que la zone ombrée s'élargit progressivement de gauche à droite.

Cette visualisation démontre que le modèle ne se contente pas de faire une simple régression ; il quantifie fidèlement l'incertitude épistémique et aléatoire. Plus la prédiction avance dans le temps (ou plus les données sont bruitées), plus le modèle indique, à juste titre, une perte de certitude en élargissant son intervalle de confiance.

4.5 Conclusion et Perspectives

L'implémentation et l'analyse de ce code nous ont permis de démystifier l'intégration de l'incertitude dans les modèles prédictifs séquentiels. À travers la méthode proposée par Alaa and van der Schaar [2020], nous avons pu constater qu'il est tout à fait possible de transformer un réseau de neurones récurrent (**RNN/LSTM**) purement déterministe en un modèle capable d'évaluer sa propre fiabilité.

4.5.1 Bilan de l'approche par Fonctions d'Influence

L'utilisation des Fonctions d'Influence pour quantifier l'incertitude fréquentiste s'avère être une stratégie redoutablement efficace. En exploitant la matrice Hessienne inverse et les gradients du modèle entraîné, cette approche *post-hoc* évite le coût prohibitif des méthodes de réentraînement systématique ou de l'inférence bayésienne. Les tests réalisés sur notre jeu de données ont visuellement prouvé que le modèle parvient à élargir ses intervalles de confiance de manière pertinente face à une incertitude croissante.

4.5.2 Limites et transition vers le Temporal Fusion Transformer (TFT)

Bien que cette méthode soit brillante mathématiquement, elle reste une "surcouche" ajoutée a posteriori sur une architecture classique. De plus, le calcul et l'inversion de la matrice Hessienne peuvent devenir des goulets d'étranglement calculatoires pour des modèles comportant des millions de paramètres.

C'est pourquoi l'évolution naturelle de notre étude se tourne vers le **Temporal Fusion Transformer (TFT)**. Contrairement aux **LSTM** complétés par des fonctions d'influence, le **TFT** propose un changement de paradigme fondamental : l'incertitude y est native. En remplaçant l'erreur quadratique moyenne (**MSE**) par une fonction de perte basée sur les quantiles (*Quantile Loss*), le **TFT** apprend organiquement à générer non pas une courbe unique, mais un faisceau de prédictions (ex : les percentiles 10, 50 et 90).

En définitive, l'étude de l'incertitude fréquentiste sur les **RNN** constitue un socle théorique indispensable. Elle met en lumière les défis mathématiques de la prévision robuste et justifie pleinement la transition vers des architectures basées sur l'attention comme le **TFT**, qui intègrent cette robustesse directement dans leur conception, répondant ainsi aux exigences des systèmes de prise de décision critiques.

5 Temporal Fusion Transformer : TFT

Le **Temporal Fusion Transformer (TFT)** est une architecture de réseau neuronal spécialisée pour la prévision interprétable de séries temporelles multi-horizons. Le TFT propose une conception modulaire unifiant les réseaux neuronaux récurrents et les mécanismes d'attention avec une sélection et un contrôle explicites des variables, ce qui lui permet de modéliser des dépendances temporelles complexes, d'intégrer des caractéristiques d'entrée hétérogènes (covariables statiques, futures connues et passées observées) et d'offrir une interprétabilité détaillée. Son architecture pallie les limitations des méthodes d'apprentissage profond antérieures, notamment leur nature opaque, en fournissant des informations sur l'importance des caractéristiques et les tendances temporelles. Le TFT atteint des performances de pointe sur divers jeux de données réels et prend en charge des tâches telles que la détection de changements de régime et l'attribution de caractéristiques, essentielles dans des applications allant de la prévision de la demande de détail à l'analyse de la volatilité financière (Lim et al., 2019). Source : Emergent Mind [2025] et Lim et al. [2021].

Composition architecturale et modules clés

L'architecture du TFT se distingue par son pipeline de modélisation de séquence encodeur-décodeur composé de plusieurs modules critiques.

Traitement local via encodeur-décodeur récurrent

Le TFT utilise des couches séquence-à-séquence basées sur des LSTM pour traiter les entrées historiques sur la fenêtre de rétrospective. Ceci permet de capturer les dynamiques temporelles à court terme et les motifs locaux. Pour l'historique des entrées $\phi(t, n)$ portées sur l'intervalle $\phi(t, -k) : \phi(t, \tau_{\max})$, la sortie du LSTM à l'instant $t + n$ est contrôlée et normalisée selon :

$$\text{LayerNorm} \left(\tilde{\xi}_{t+n} + \text{GLU}(\phi(t, n)) \right)$$

ce qui permet des connexions résiduelles contrôlées entre les états du LSTM et leurs entrées.

Couche d'auto-attention multi-têtes

Appliquée après les couches récurrentes, cette couche d'attention interprétable capture les dépendances à long terme, permettant au modèle de prendre en compte les relations sur des intervalles de temps arbitraires. Le mécanisme d'attention est défini comme :

$$\text{InterpretableMultiHead}(Q, K, V) = \left[\frac{1}{H} \sum_{h=1}^H \text{Attention} \left(QW_Q^{(h)}, KW_K^{(h)}, VW_V \right) \right] W_H$$

avec l'attention produit scalaire normalisée :

$$\text{Attention}(Q, K, V) = \text{Softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V$$

La transformation de la valeur VW_V est partagée entre les différentes têtes, favorisant ainsi l'interprétabilité grâce à la possibilité d'avoir une vue globale des pondérations d'attention.

Réseaux de sélection de caractéristiques

Des modules de sélection de variables distincts sont appliqués aux entrées statiques, passées et futures. Chaque variable d'entrée $x_t^{(j)}$ est projetée dans un espace latent partagé et traitée par un réseau résiduel à portes (GRN).

Les transformations sont données par :

$$\xi_t^{(j)} \rightarrow \text{GRN}(\xi_t^{(j)})$$

et les poids de sélection des variables sont obtenus par :

$$v_{\chi_t}^{(j)} = \text{Softmax}(\text{GRN}_v(\Xi_t, c_s))$$

L'agrégation produit alors un vecteur de contexte unique :

$$\tilde{\xi}_t = \sum_j v_{\chi_t}^{(j)} \cdot \tilde{\xi}_t^{(j)}$$

permettant une priorisation dynamique des caractéristiques à chaque étape temporelle.

Mécanismes de régulation

Les unités linéaires à porte (GLU) et les réseaux résiduels à portes (GRN) régulent dynamiquement le flux d'information. Un GRN générique est formulé comme :

$$\text{GRN}_\omega(a, c) = \text{LayerNorm}(a + \text{GLU}_\omega(\eta_1))$$

où η_1 et η_2 représentent les transformations internes de non-linéarité et de porte.

Sélection des fonctionnalités et intégration des entrées

La gestion de l'information dans le TFT repose sur un traitement sélectif permettant à la fois une forte expressivité et une bonne interprétabilité.

Pour chaque groupe d'entrée (statique, passé observé, futur connu), un réseau de sélection de variables projette et évalue individuellement chaque variable. Pour les entrées temporelles :

$$\xi_t^{(j)} \rightarrow \text{GRN}(\xi_t^{(j)}) \rightarrow v_{\chi_t}^{(j)} = \text{Softmax}(\text{GRN}_v(\Xi_t, c_s))$$

Le vecteur d'entrée final est obtenu par :

$$\tilde{\xi}_t = \sum_j v_{\chi_t}^{(j)} \tilde{\xi}_t^{(j)}$$

Toutes les transformations clés sont régulées par des GLU dont les coefficients peuvent annuler certaines caractéristiques ou voies du réseau, permettant une profondeur sélective et une adaptation à la complexité du signal.

Interprétabilité

L'interprétabilité constitue une composante fondamentale du TFT.

Les pondérations issues des modules de sélection de variables quantifient directement l'importance des caractéristiques pour chaque instance et chaque instant.

Les matrices d'attention de sortie

$$\alpha(t, n, \tau)$$

permettent d'analyser quels instants historiques influencent chaque horizon de prévision.

Le TFT peut quantifier les changements de régime grâce à une métrique basée sur le coefficient de Bhattacharyya entre vecteurs d'attention :

$$\kappa(p, q) = 1 - \rho(p, q)$$

avec

$$\rho(p, q) = \sum_j \sqrt{p_j q_j}$$

Les pics de cette métrique correspondent à des périodes de changements dynamiques importants, comme observé lors de certains événements économiques majeurs tels que la crise financière de 2008.

Applications et cas d'utilisation multi-domaines

La flexibilité de gestion des entrées et l'interprétabilité du TFT permettent son application dans de nombreux contextes réels.

Dans le domaine de la prévision de la consommation d'électricité, le modèle combine les observations passées et les caractéristiques calendaires connues afin de capturer les cycles journaliers et saisonniers.

Pour la surveillance du trafic routier, il exploite plusieurs signaux dynamiques afin de prédire l'occupation du réseau et d'identifier les dépendances temporelles locales et globales.

Dans le secteur du commerce de détail, il intègre des métadonnées statiques liées aux magasins ou aux produits ainsi que des événements exogènes tels que les promotions ou les jours fériés afin de prévoir les ventes.

Enfin, dans le domaine de la volatilité financière, les modules d'interprétabilité permettent d'identifier les périodes critiques de changement de régime sur des séries de données bruitées et fortement non stationnaires.

Importance technique et principes de conception

L'architecture modulaire du TFT repose sur plusieurs principes fondamentaux.

Premièrement, l'encodage explicite de l'hétérogénéité des entrées permet de traiter séparément les variables statiques, dynamiques et futures connues, ce qui améliore l'interprétabilité et permet d'ignorer dynamiquement les signaux non pertinents.

Deuxièmement, la combinaison d'un mécanisme récurrent local et d'un mécanisme d'attention global permet d'apprendre simultanément les dépendances à court et à long terme.

Troisièmement, l'utilisation de mécanismes de régulation tels que les portes et le dropout permet une adaptation contrôlée de la profondeur du modèle et de sa capacité de représentation.

Enfin, grâce à cette conception, le TFT fournit non seulement des prévisions précises mais également des explications directes concernant les variables et les périodes temporelles qui influencent ces prévisions.

5.1 Description : Un modèle hybride de prédiction par quantiles

En tant que travail préliminaire, nous avons évalué un modèle LSTM pur accompagné du système de quantification basé sur l'approche par fonction d'influence (ibid) développée par Ahmed M. Alaa et Mihaela van der Schaar (Ahmed M Alaa, 2020). Cette méthode de calcul d'incertitude a pour avantage de ne pas nécessiter une modification de l'architecture du modèle ni de son entraînement (contrairement aux approches des RNN Bayésien et RNN par Quantile qui sont plus lourdes à mettre en place) tout en apportant un intervalle de confiance pour chaque prédiction de valeurs.

Cependant, bien que notre LSTM ait été fonctionnel, le système de quantification d'incertitude était beaucoup trop gourmand et n'était pas applicable pour notre masse de données financières. Cette première expérience nous a amené à chercher un autre système plus compatible avec notre étude. Nous nous sommes tournés vers une solution plus complexe que prévu mais beaucoup plus pertinente pour ce travail. Pour notre étude nous utilisons le Temporal Fusion Transformer (Bryan Lima, 2020). Ce modèle, développé par Google et Oxford fait partie de ces nouveaux modèles hybrides qui mélangent Deep Learning et méthodes statistiques depuis l'ES-RNN d'Uber. En ayant battu le modèle de prédiction DeepAR d'Amazon qui utilise une série de LSTM, le Temporal Fusion Transformer (appelé TFT) est aujourd'hui le nec plus ultra des modèles de prédiction de séries chronologiques à multiple horizons temporels. Ses hautes performances proviennent d'une architecture complexe qui mélange deux LSTM, des GRN (Gated Residual Network ou Réseau Résiduel à Portes), des réseaux de sélection de variables et un modèle d'attention.

Contrairement à la plupart des modèles hybrides, le TFT est capable d'étudier des données statiques en plus d'une série chronologique. Plus précisément, ce dernier va pouvoir affiner ses prédictions en prenant en compte des données exogènes en lien avec série chronologique que nous aurons renseigné. Ces données peuvent être : les jours fériés, la moyenne des valeurs d'ouverture sur 10 jours, l'augmentation de la valeur d'une action sur une même journée, etc. Cette fonctionnalité peut s'avérer déterminante lorsque nous essayons de modéliser l'évolution de la valeur d'une action puisque beaucoup de paramètres exogènes peuvent avoir un rôle sur ces fluctuations. Cependant, il convient de les choisir avec pertinence, nous ne pouvons inclure toutes les données en lien avec le produit financier : au mieux, la construction d'une telle base de données serait un travail gigantesque. Au pire, nous risquons de noyer quelques éléments pertinents dans une masse d'information triviale. Il convient donc de sélectionner minutieusement les données exogènes fournies. Notre prochaine sous-partie détaillera ce processus. Finalement, le TFT embarque un système personnalisé de quantification d'incertitude par quantile. Pour chaque quantile défini, le TFT calcule la prédiction. Cela nous permet d'utiliser directement le modèle sans modifications structurelles qui dépassera la modification nécessaire à sa mise en place. Autrement dit, nous avons la certitude d'utiliser un modèle de quantification d'incertitude totalement adapté et optimisé pour notre modèle de prédiction. Cependant, concernant ce système de quantification d'incertitude, il est important de détailler clairement ses limites avant l'application. Le TFT utilise des régressions par

quantile pour produire ses prédictions, toutefois, cette méthode bien que très performante lorsqu'elle est fonctionnelle, n'est pas sans erreurs. De manière générale, les prédictions par quantiles nous laissent penser que nous maîtrisons totalement le risque alors qu'il peut s'avérer que ces quantiles ne soient pas significatifs. Ce phénomène augmente plus les données diminuent.

5.1.1 Fonctionnement du code

Dans cette sous-section, nous détaillerons chaque bloc de code nécessaire à la mise en place du modèle de TFT.

Chargement des données et *Feature Engineering*

Après l'importation des modules nécessaires à notre étude, la première étape de notre démarche correspond au chargement des données financières brutes et au processus de *Feature Engineering* (ingénierie des caractéristiques). Ce procédé consiste à créer de nouvelles variables synthétiques (ou variables exogènes) à partir de notre historique de prix. Cette partie vise à enrichir l'information fournie au modèle et à évaluer l'importance de ces nouveaux indicateurs sur la qualité globale des prédictions. Dans cette partie du code TFT, On crée plusieurs entrées de variables toutes attribuées à une catégorie. Ces catégories sont les suivantes :

- **Variables principales :**
 - **Variable cible :** la variable que nous allons essayer de prédire.
 - **Variable d'index temporel :** la variable qui marque l'évolution chronologique.
 - **Variables de groupement :** les variables qui permettent de regrouper les données.
- **Variables complémentaires :**
 - **Variables statiques de catégorie :** variables complémentaires de la variable cible, sous forme catégorielle (ex. : l'année de la variable cible).
 - **Variables statiques réelles :** variables complémentaires de la variable cible, sous forme réelle.
 - **Variables chronologiques connues de catégorie :** variables connues avant chaque prédiction, sous forme catégorielle (ex. : le jour ; nous connaissons le jour de la cible avant d'émettre une prédiction).
 - **Variables chronologiques inconnues de catégorie :** variables non connues avant chaque prédiction, sous forme catégorielle (ex. : `day_up` qui affiche 1 si le prix du produit étudié connaît une progression à l'indice temporel t).
 - **Variables réelles chronologiques connues :** variables connues avant chaque prédiction et variant dans le temps.
 - **Variables réelles chronologiques inconnues :** variables non connues avant chaque prédiction et variant dans le temps (ex. : valeur de fermeture, `log_open`, etc.).

Parmi les variables créées, nous retrouvons notamment celles calculant la moyenne des X dernières valeurs d'ouverture : `avg_last_5_days` et `avg_last_10_days`. Celles-ci ont été intégrées au modèle en tant que variables statiques réelles. Elles permettent au modèle de capter la tendance de très court terme, en complément des dépendances chronologiques qu'il incorpore naturellement. En définissant ces variables, nous contraignons le modèle à établir un lien entre ces moyennes et ses prédictions.

Concernant la variable `cat_month_temporal_code`, celle-ci associe à chaque mois un identifiant numérique unique. Par exemple, le mois de janvier 2017 ne possède pas le même identifiant que celui des autres mois. C'est grâce à la variable `cat_month_temporal_code` que le TFT est capable de regrouper les données par mois et ainsi de fonctionner selon la structure que nous avons définie. Techniquement, à partir de cette variable, le modèle utilise les trois quarts des observations d'un mois afin de prédire le quart restant. Cette variable a donc été introduite en tant que variable de regroupement.

Concernant la variable `log_open`, celle-ci correspond à la transformation logarithmique de notre variable cible `open`. De la même manière qu'il peut être pertinent, dans un modèle de régression linéaire, de transformer la variable à expliquer afin d'améliorer les performances ou la stabilité du modèle, un modèle de Deep Learning peut également bénéficier d'une telle transformation. Cette variable a ainsi été intégrée comme variable réelle chronologique inconnue.

Nous avons également créé une variable binaire nommée `day_up`. Celle-ci prend la valeur 1 lorsque la valeur de fermeture du produit est supérieure à sa valeur d'ouverture pour un jour donné. Elle fournit une indication forte sur l'évolution journalière et permet, à travers une seule variable, de capturer la tendance du dernier jour. Après transformation en variable catégorielle, elle a été intégrée en tant que variable chronologique inconnue de catégorie, car sa valeur dépend directement de la variable cible `open`. Nous avons décliné cette variable en une seconde, `last_was_day_up`, qui informe le modèle sur la valeur de `day_up` à la temporalité `t-1`. Cette variable permet ainsi d'intégrer explicitement l'information relative à la tendance de la veille. Étant donné qu'elle concerne une temporalité passée, sa valeur est connue au moment de la prédiction pour la temporalité `t`. Elle est donc classée parmi les variables catégorielles connues. Certaines autres variables correspondent uniquement à des transformations de type : nous convertissons des variables initialement encodées sous forme de chaînes de caractères en variables catégorielles afin de les rendre exploitables par le modèle. L'ensemble des variables décrites ici, ainsi que d'autres détaillées en annexe, est automatiquement généré par notre algorithme pour chacune des séries chronologiques considérées. Elles sont ensuite injectées dans le système pour chaque série.

Une fois la série de données complète, celle-ci est transmise à la méthode `TimeSeriesDataSet`, en précisant les différentes catégories de variables définies. Cette méthode assure la validation des données ainsi que leur transformation en vecteurs adaptés au fonctionnement du Temporal Fusion Transformer. Enfin, le modèle permet d'analyser l'importance relative de ces variables grâce à son mécanisme d'interprétation post-entraînement. Cet aspect sera développé dans la section suivante.

```

1 # 1. Télécharger les données de l'action Apple (symbole "AAPL")
2 data = yf.download("AAPL", start="2011-01-01", end="2019-01-01")
3
4 # On supprime le deuxième niveau de nommage des colonnes (le symbole "
  AAPL")
5 if isinstance(data.columns, pd.MultiIndex):
6     data.columns = data.columns.droplevel(1)
7

```

```

8 print(data)
9
10 #####
11 print('='*50)
12 print('Feature Ingeneering')
13 print('='*50)
14 #####
15
16 data = data.reset_index()
17
18 # add time index
19 data["time_idx"] = data.index
20
21 # add additional features
22 data['Date'] = pd.to_datetime(data['Date'], errors='coerce')
23 data["cat_month"] = data["Date"].dt.month.astype(str).astype("category")
24 data["cat_year"] = data["Date"].dt.year.astype(str).astype("category")
25 data["cat_day"] = data["Date"].dt.day.astype(str).astype("category")
26
27 data["year_code"] = (data.cat_year.astype(int) - data.cat_year.astype(
    int).min())
28 data["cat_year_code"] = data.year_code.astype("category")
29 data["month_temporal_code"] = (data.cat_month.astype(int) + data.year_
    code * 12)
30 data["cat_month_temporal_code"] = data.month_temporal_code.astype("
    category")
31
32 data["log_open"] = np.log(data.Open + 1e-8)
33 data["close_open"] = data.Close - data.Open
34
35 # 1 si l'action monte dans la journee, 0 sinon
36 data["day_up"] = np.where(data.close_open > 0, 1, 0)
37 data["cat_day_up"] = data.day_up.astype(str).astype("category")
38
39 # 1 si la veille etait positive
40 data["last_was_day_up"] = np.where(data.close_open.shift(1) > 0, 1, 0)
41 data["cat_last_was_day_up"] = data.last_was_day_up.astype(str).astype("
    category")
42
43 # Ouverture superieure a la veille ?
44 data["open_up"] = np.where(data.Open > data.Open.shift(1), 1, 0)
45 data["cat_open_up"] = data.open_up.astype(str).astype("category")
46
47 # Moyennes mobiles sur les 5 et 10 derniers jours
48 data["avg_last_5_days"] = (
49     data.Open.shift(1) + data.Open.shift(2) +
50     data.Open.shift(3) + data.Open.shift(4) +
51     data.Open.shift(5)
52 ) / 5
53
54 data["avg_last_10_days"] = (
55     data.Open.shift(1) + data.Open.shift(2) +
56     data.Open.shift(3) + data.Open.shift(4) +
57     data.Open.shift(5) + data.Open.shift(6) +
58     data.Open.shift(7) + data.Open.shift(8) +
59     data.Open.shift(9) + data.Open.shift(10)
60 ) / 10
61

```

```
62 # On retire les dix premières lignes à cause du calcul des moyennes  
    mobiles sur les 10 derniers jours  
63 data = data.iloc[10:]
```

Structuration temporelle et configuration du jeu de données

Cette étape du code est entièrement consacrée à la structuration temporelle de nos données afin de les rendre lisibles par le Temporal Fusion Transformer. Nous définissons tout d'abord les fenêtres d'apprentissage et de prédiction de notre intelligence artificielle. Le modèle est configuré pour observer un historique strict de dix-sept jours consécutifs, correspondant à la taille de l'encodeur, dans le but de prédire une fenêtre d'horizon allant de trois à cinq jours maximum dans le futur. Pour s'assurer que le modèle puisse être évalué de manière totalement impartiale à la fin de son apprentissage, une limite temporelle de coupure est calculée. Cette limite permet d'exclure les tout derniers jours de notre historique, qui seront ainsi mis de côté et strictement réservés à la phase de validation.

Le cœur de cette préparation repose ensuite sur l'initialisation de l'objet `TimeSeriesDataSet`. Cet outil puissant va transformer notre tableau de données classique en une structure matricielle complexe, indispensable pour le Deep Learning. Nous lui indiquons explicitement que notre objectif de prédiction est le prix d'ouverture, tout en lui demandant de regrouper nos séquences de manière mensuelle. Une étape fondamentale de cette configuration, spécifique et obligatoire pour l'architecture de ce modèle précis, consiste à classer rigoureusement chaque variable de notre base de données selon sa nature et sa disponibilité dans le temps.

Les variables sont ainsi réparties dans différentes catégories temporelles afin d'éviter toute fuite de données futures. Nous définissons d'abord les variables considérées comme statiques pour une séquence donnée, incluant nos moyennes mobiles lissées sur plusieurs jours et certaines identifications catégorielles. Nous déclarons ensuite les variables dynamiques connues à l'avance, c'est-à-dire les informations calendaires ou d'indexation dont la valeur future est une certitude mathématique. Nous isolons enfin les variables dynamiques inconnues, qui regroupent absolument toutes les données de marché imprévisibles dans le futur, comme les prix hauts et bas, les volumes échangés ou la direction journalière du marché. Cette classification stricte garantit que le modèle apprendra uniquement à partir des informations dont il disposerait réellement en conditions réelles.

La dernière phase de ce bloc de code finalise la mise en forme avant le lancement des calculs. Les données cibles subissent une normalisation par groupe, une technique permettant d'harmoniser les échelles de valeurs et d'accélérer l'apprentissage du réseau de neurones. À partir de cette configuration d'entraînement initiale, le code génère automatiquement le jeu de validation jumeau, paramétré spécifiquement pour évaluer les prédictions sur la toute dernière période temporelle cachée au modèle. L'ensemble de ces données est ultimement encapsulé dans des chargeurs de données, appelés `Dataloaders`. Ces derniers ont pour rôle essentiel de regrouper les historiques par paquets de cent vingt-huit séquences, permettant d'alimenter le processeur de manière fluide, itérative et optimisée lors de la phase d'entraînement de la machine.

```

1 # Parametres du dataset pour l'entrainement
2 max_prediction_length = 5 # maximum 5 jours de donnees    predire
3 max_encoder_length = 17 # maximum 17 jours de donnees entrantes
4 training_cutoff = data["time_idx"].max() - max_prediction_length
5
6 # Creation du training
7 training = TimeSeriesDataSet(
8     data[lamba x: x.time_idx <= training_cutoff],
9     time_idx="time_idx", # notre index temporel
10    target="Open", # ce que nous essayons de predire: le prix d'
        ouverture
11    group_ids=["cat_month_temporal_code"], # regroupement par mois
12
13    min_encoder_length=max_encoder_length,
14    max_encoder_length=max_encoder_length,
15
16    min_prediction_length=3, # au minimum, on predit 3 jours
17    max_prediction_length=max_prediction_length,
18
19    static_categoricals=["cat_year", "cat_month", "cat_last_was_day_up"
20                        ],
21    static_reals=["avg_last_5_days", "avg_last_10_days"],
22
23    time_varying_known_categoricals=["cat_year", "cat_month"],
24    variable_groups={},
25
26    time_varying_known_reals=["time_idx", "year_code", "month_temporal_
27                             code"],
28    time_varying_unknown_categoricals=["cat_day_up", "cat_open_up"],
29
30    time_varying_unknown_reals=[
31        "Open", "log_open", "Volume", "High", "Low", "Close"
32    ],
33
34    target_normalizer=GroupNormalizer(),
35    add_relative_time_idx=True,
36    add_target_scales=True,
37    add_encoder_length=True,
38 )
39
40 # Create validation set
41 validation = TimeSeriesDataSet.from_dataset(
42     training, data, predict=True, stop_randomization=True
43 )
44
45 # Create dataloaders
46 batch_size = 128 # entre 32 et 128
47 train_dataloader = training.to_dataloader(
48     train=True, batch_size=batch_size, num_workers=0
49 )
50
51 val_dataloader = validation.to_dataloader(
52     train=False, batch_size=batch_size * 10, num_workers=0
53 )

```

Listing 11 – Creation du dataset et des dataloaders

Établissement du modèle de référence (Baseline) et calcul de l'erreur

Avant d'entamer la phase d'apprentissage complexe de notre réseau de neurones, cette courte étape de code est incontournable pour établir un point de comparaison élémentaire, souvent qualifié de modèle de référence ou de "Baseline". L'objectif est d'évaluer la pertinence d'une prédiction qualifiée de naïve, dont le postulat est extrêmement simple : la valeur de demain sera strictement identique à la dernière valeur connue d'aujourd'hui.

Pour réaliser cette évaluation, le code procède en trois étapes distinctes. Il commence tout d'abord par extraire l'ensemble des valeurs réelles issues de notre jeu de validation. Ces données correspondent à la véritable évolution du marché durant les derniers jours de la série temporelle, ceux-là mêmes qui ont été volontairement soustraits à la vue du modèle pour garantir l'intégrité du test. Dans un second temps, le script fait appel au modèle de référence paresseux et lui demande de générer ses propres prédictions statiques sur cette même fenêtre temporelle.

Enfin, la dernière instruction procède à la confrontation mathématique entre ces deux séries de données afin de calculer l'Erreur Absolue Moyenne, plus connue sous son acronyme anglophone MAE (Mean Absolute Error). Ce calcul s'effectue en mesurant d'abord la différence absolue entre chaque prédiction naïve et la réalité du marché, évitant ainsi que des erreurs positives et négatives ne s'annulent entre elles, puis en effectuant la moyenne globale de ces écarts. La valeur numérique finale qui en résulte est fondamentale pour la suite de notre étude : elle constitue notre seuil de rentabilité prédictive. Pour que notre architecture complexe, le Temporal Fusion Transformer, prouve sa réelle utilité et sa capacité d'anticipation, il devra impérativement obtenir une erreur moyenne finale significativement inférieure à celle de cette simple répétition du passé.

```

1 # ENG: calculate baseline mean absolute error,
2 # i.e. predict next value as the last available value from the history
3
4 # Extraction des vraies valeurs (jeu de validation)
5 actuals = torch.cat(
6     [y for x, (y, weight) in iter(val_dataloader)]
7 )
8
9
10 baseline_predictions = Baseline().predict(val_dataloader)
11
12 # Calcul de la MAE (Mean Absolute Error)
13 mae = (actuals - baseline_predictions).abs().mean().item()
14
15 mae

```

Listing 12 – Calcul de la baseline et de la MAE

Configuration de l'apprentissage et architecture du TFT

Cette section du code est dédiée à la configuration du moteur d'entraînement et à l'initialisation de l'architecture complexe de notre modèle d'intelligence artificielle. Avant de

lancer les calculs, des mécanismes de sécurité rigoureux sont mis en place, dont le plus critique est l'arrêt prématuré, ou « Early Stopping ». Ce filet de sécurité surveille en continu l'erreur du modèle sur le jeu de données de validation. Si cette erreur cesse de s'améliorer pendant dix cycles d'apprentissage consécutifs, le processus est immédiatement et automatiquement interrompu. Cette précaution est absolue et indispensable en apprentissage profond pour éviter le phénomène de surapprentissage (overfitting). Sans cela, le réseau de neurones risquerait de mémoriser les données historiques par cœur au lieu d'en dégager des tendances générales, ce qui le rendrait totalement incapable de généraliser et de prédire l'avenir. Ces règles de sécurité, associées à des outils de journalisation des performances, sont ensuite confiées au superviseur du programme, appelé le « Trainer ». Ce dernier est rigoureusement paramétré pour gérer l'accélération matérielle de l'ordinateur, limiter la durée maximale de l'entraînement à cent époques, et contrôler mathématiquement l'explosion des gradients.

Une fois le cadre d'apprentissage solidement défini, la seconde partie du script procède à la création du cerveau analytique lui-même : le Temporal Fusion Transformer (TFT). Ce réseau de neurones est instancié en se basant directement sur la structure de notre jeu d'entraînement préalablement formaté. L'architecture interne du modèle est régie par une série d'hyperparamètres d'une précision millimétrée, régulant notamment le taux d'apprentissage, la taille des couches cachées, le nombre de têtes d'attention, ainsi que le taux d'oubli volontaire (dropout) qui participe activement à la robustesse du réseau. La présence de valeurs décimales aussi spécifiques démontre que ces paramètres n'ont pas été fixés arbitrairement, mais qu'ils sont le fruit d'un algorithme d'optimisation préalable ayant testé de multiples combinaisons pour retenir la plus performante.

Enfin, la caractéristique la plus novatrice et centrale de ce code par rapport à notre problématique de recherche réside dans l'utilisation de la fonction de perte par quantiles (« Quantile Loss ») couplée à une dimension de sortie fixée à sept. Contrairement aux architectures classiques, telles que les simples réseaux LSTM, qui se contentent de prédire un point unique pour le futur, notre modèle est mathématiquement contraint de générer sept trajectoires distinctes. Cette modélisation probabiliste permet au réseau de ne plus délivrer une simple estimation médiane, mais d'assortir sa prédiction de plusieurs intervalles de confiance. C'est précisément ce mécanisme qui permet de quantifier visuellement et mathématiquement l'incertitude du marché face aux événements futurs.

```
1 # Configuration du modele et de l'entrainement
2
3 early_stop_callback = EarlyStopping(
4     monitor="val_loss",
5     min_delta=1e-4,
6     patience=10,
7     verbose=False,
8     mode="min"
9 )
10
11 # Si pendant 10 epochs consecutives l'erreur ne s'ameliore plus, on
   arrete
12 lr_logger = LearningRateMonitor()
13 logger = TensorBoardLogger("lightning_logs")
```

```

14
15 trainer = pl.Trainer(
16     max_epochs=100,
17     accelerator="auto",
18     devices=1,
19     gradient_clip_val=0.6593429242999529,
20     limit_train_batches=30,
21     callbacks=[lr_logger, early_stop_callback],
22     logger=logger,
23 )
24
25 tft = TemporalFusionTransformer.from_dataset(
26     training,
27     learning_rate=0.06958191715695854,
28     hidden_size=84,
29     attention_head_size=2,
30     dropout=0.1259586140697932,
31     hidden_continuous_size=26,
32     output_size=7, # 7 quantiles par défaut
33     loss=QuantileLoss(),
34     log_interval=10,
35     reduce_on_plateau_patience=4,
36 )
37
38 print(f"Number of parameters in network: {tft.size()/1e3:.1f}k")

```

Listing 13 – Configuration du modèle Temporal Fusion Transformer

Exécution de l'apprentissage du réseau de neurones

Cette étape représente l'aboutissement de toutes les phases de configuration antérieures. L'instruction exécutée déclenche le processus d'apprentissage effectif du Temporal Fusion Transformer. Durant cette phase de calcul, le modèle parcourt de manière itérative les paquets de données fournis par le chargeur d'entraînement afin d'ajuster ses poids mathématiques internes et de minimiser progressivement son erreur. Simultanément, à la fin de chaque cycle d'apprentissage, ou époque, le système confronte immédiatement les nouvelles connaissances du réseau à notre jeu de données de validation.

Ce va-et-vient constant entre l'assimilation du passé et la vérification sur des données cachées permet au superviseur d'évaluer la performance de la machine en temps réel. C'est précisément lors de cette exécution que les mécanismes de sécurité définis à l'étape précédente, en particulier l'arrêt prématuré, prennent tout leur sens en stoppant net le processus si les performances stagnent. Bien que cette commande soit d'une extrême concision d'un point de vue syntaxique, elle déclenche la phase la plus exigeante en ressources de notre programme. Elle mobilise la puissance de calcul du processeur graphique durant de longues minutes afin de réaliser les millions d'opérations matricielles indispensables à l'élaboration de nos futures prédictions probabilistes.

```

1 # Entraînement du modèle
2
3 trainer.fit(
4     tft,

```



```

5     train_dataloaders=train_dataloader ,
6     val_dataloaders=val_dataloader ,
7 )

```

Listing 14 – Entraînement du modèle

Optimisation des hyperparamètres par recherche algorithmique

Cette section du code illustre une phase fondamentale en apprentissage profond : la recherche des hyperparamètres optimaux. Plutôt que de définir arbitrairement l'architecture interne du Temporal Fusion Transformer et la vitesse à laquelle il doit apprendre, nous délégons cette tâche complexe à un algorithme d'optimisation automatisé, très souvent basé sur la bibliothèque Optuna. Le rôle de ce script est de lancer une véritable campagne d'expérimentation scientifique durant laquelle l'ordinateur va tester de lui-même plusieurs combinaisons de réglages mathématiques afin de déterminer empiriquement laquelle génère les prédictions les plus précises sur nos données de validation.

Pour guider intelligemment cette recherche, nous définissons rigoureusement un espace d'exploration sous forme de limites minimales et maximales. Nous indiquons à l'algorithme les plages de valeurs autorisées pour les composants clés du modèle. Cela inclut notamment la taille des couches cachées qui détermine la capacité de compréhension du réseau, le nombre de têtes d'attention pour capter les signaux du passé, le taux d'apprentissage qui dicte la vitesse d'assimilation des erreurs, ainsi que le taux d'oubli volontaire, communément appelé dropout, servant à contraindre le modèle pour éviter le surapprentissage. Afin de maîtriser le temps de calcul informatique, qui peut rapidement devenir prohibitif, cette procédure de recherche est paramétrée pour s'arrêter au bout d'un nombre d'essais défini, tout en limitant la quantité de paquets de données ingérés lors de chaque test éphémère.

À l'issue de cette exploration automatisée, le programme compare minutieusement les performances de chaque architecture testée. La toute dernière instruction de ce bloc de code a pour but d'isoler, de sauvegarder et d'afficher à l'écran la combinaison exacte des paramètres vainqueurs. Ces réglages optimaux, sélectionnés de manière purement mathématique par la machine comme étant les plus performants, constitueront l'architecture définitive et sur-mesure de notre Temporal Fusion Transformer lors de la phase finale de notre projet.

```

1 from pytorch_forecasting.models.temporal_fusion_transformer.tuning
  import optimize_hyperparameters
2
3 # Creation de l'etude d'optimisation
4 study = optimize_hyperparameters(
5     train_dataloader,
6     val_dataloader,
7     model_path="optuna_test",
8     n_trials=5, # 5 essais pour test
9     max_epochs=100,
10
11     gradient_clip_val_range=(0.01, 1.0),
12     hidden_size_range=(8, 128),
13     hidden_continuous_size_range=(8, 128),

```

```

14     attention_head_size_range=(1, 4),
15     learning_rate_range=(0.001, 0.1),
16     dropout_range=(0.1, 0.3),
17
18     trainer_kwargs=dict(
19         limit_train_batches=30,
20         accelerator="auto",
21         devices=1
22     ),
23
24     reduce_on_plateau_patience=4,
25     use_learning_rate_finder=False,
26 )
27
28 # Affichage des meilleurs hyperparametres
29 print("=== Meilleurs parametres trouves ===")
30 print(study.best_trial.params)

```

Listing 15 – Optimisation des hyperparamètres avec Optuna

Extraction et restauration du modèle optimal

Une fois la phase d'apprentissage totalement terminée, cette courte section du code intervient pour récupérer la version la plus performante de notre intelligence artificielle. Il est fondamental de comprendre qu'en apprentissage profond, le dernier modèle généré à la toute fin du processus d'entraînement n'est pas systématiquement le meilleur. En effet, au fil des itérations, le réseau de neurones peut commencer à apprendre les données par cœur et perdre sa capacité de généralisation, ce qui dégrade ses performances sur le jeu de données de validation. Heureusement, grâce aux mécanismes de sauvegarde automatisés mis en place par l'environnement PyTorch Lightning, le système a enregistré en arrière-plan l'état exact du modèle au moment précis où son taux d'erreur était à son niveau le plus bas absolu.

Le script commence donc par interroger le superviseur de l'entraînement pour récupérer le chemin d'accès informatique menant au fichier de sauvegarde de cette version idéale. Ensuite, la seconde instruction utilise ce chemin pour recharger complètement l'architecture du Temporal Fusion Transformer dans son état de performance maximale. Cette étape est cruciale pour la rigueur scientifique de l'étude : elle garantit que c'est exclusivement ce modèle optimal, mathématiquement validé comme étant le plus robuste face à des données inconnues, qui sera utilisé pour générer l'ensemble de nos prédictions probabilistes finales.

Évaluation finale et calcul de l'erreur du modèle optimal

Cette dernière étape analytique vient clore la phase d'apprentissage en chiffrant concrètement la performance mathématique de notre réseau de neurones. L'objectif de ces instructions est de calculer l'Erreur Absolue Moyenne, ou MAE, de notre modèle optimal sur le jeu de données de validation. Cette démarche est strictement identique à celle réalisée précédemment pour notre modèle de référence (la Baseline), car le but ultime de ce projet est de comparer ces deux scores afin de valider ou d'invalidier l'utilité de notre intelligence artificielle.

Le script procède de manière tout à fait méthodique. Il commence par extraire minutieusement les véritables valeurs de marché, qui étaient jusqu'alors dissimulées dans le chargeur de validation. Dans un second temps, il soumet les séquences historiques à notre meilleur Temporal Fusion Transformer, tout juste restauré, afin qu'il génère ses propres prédictions sur cet horizon temporel. Enfin, la dernière instruction confronte le fruit de cet apprentissage à la réalité du marché. Elle soustrait les prédictions de l'algorithme aux valeurs réelles, convertit ces écarts en valeurs absolues pour ne pas fausser le résultat, et en calcule la moyenne globale.

Le résultat chiffré qui s'affiche à l'issue de ce calcul représente la marge d'erreur moyenne de notre modèle final. C'est le véritable moment de vérité de notre expérimentation. Pour prouver la supériorité et la puissance de l'apprentissage profond face à une méthode statistique classique, ce score final doit impérativement être inférieur à l'erreur obtenue par le modèle paresseux. Si cette condition est remplie, cela démontre scientifiquement que l'algorithme ne s'est pas contenté d'apprendre par cœur ou de reproduire le passé, mais qu'il a bel et bien réussi à capter des signaux complexes lui permettant d'anticiper les fluctuations futures du marché.

```
1 # ENG: calculate mean absolute error on validation set
2 actuals = torch.cat([y[0] for x, y in iter(val_dataloader)])
3 predictions = best_tft.predict(val_dataloader)
4 (actuals - predictions).abs().mean()
```

Génération des prédictions probabilistes brutes

Cette instruction marque la transition entre la phase d'évaluation mathématique globale et l'extraction détaillée de nos résultats finaux. Concrètement, nous demandons à notre modèle optimal, préalablement restauré, de parcourir une dernière fois nos données de validation afin d'en déduire l'avenir. La subtilité majeure de cette ligne de code réside dans l'utilisation de l'argument spécifiant un mode de prédiction brut, appelé « raw ». Au lieu de nous renvoyer une simple liste contenant une estimation unique pour chaque jour futur, le Temporal Fusion Transformer nous restitue un objet informatique extrêmement riche. C'est précisément au sein de cette structure que se trouvent les calculs de nos sept quantiles. Cette dimension probabiliste est le cœur même de notre projet de recherche, car elle nous fournira la matière première pour tracer non seulement la trajectoire médiane la plus probable, mais surtout les marges d'incertitude encadrant cette prédiction.

Parallèlement à la génération de ces probabilités, l'activation du paramètre de retour ordonne à la fonction de nous restituer également l'historique exact des jours passés qui ont servi de base à cette déduction, stockés ici dans la variable d'abscisse. Posséder simultanément la séquence du passé et les probabilités du futur est une condition technique indispensable pour pouvoir générer nos futures représentations graphiques de manière visuellement continue. Enfin, d'un point de vue purement informatique, cette ligne intègre une syntaxe de dépaquetage flexible grâce à l'utilisation du symbole astérisque. Cette manipulation élégante permet de capturer proprement les prédictions et l'historique dans nos deux variables principales, tout en regroupant dynamiquement les éventuelles données de profilage supplémentaires dans une variable de reste, garantissant ainsi la stabilité absolue

de l'exécution du programme face aux évolutions de la bibliothèque sous-jacente.

```

1 raw_predictions, x, *le_reste = best_tft.predict(
2     val_dataloader,
3     mode="raw",
4     return_x=True
5 )

```

Génération et personnalisation des représentations graphiques

Cette dernière section du code est entièrement consacrée à la matérialisation visuelle de nos résultats, une étape indispensable pour interpréter l'efficacité de l'algorithme. Après avoir calculé les probabilités mathématiques, le script fait appel à la bibliothèque standard Matplotlib pour initialiser une zone de dessin aux dimensions contrôlées, offrant ainsi un cadre clair et lisible pour l'analyse. L'instruction centrale repose sur la méthode graphique intégrée au modèle, qui se charge de superposer automatiquement les données historiques réelles avec les prédictions générées. En figeant l'index d'observation à zéro, nous ordonnons au programme de se concentrer spécifiquement sur la toute première fenêtre temporelle de notre jeu de validation.

C'est précisément lors de l'exécution de cette commande que l'approche probabiliste de notre projet prend tout son sens visuel. Le graphique généré ne se contente pas de tracer une ligne unique pour l'avenir, mais déploie visuellement l'ensemble des sept quantiles calculés précédemment. Il représente la trajectoire médiane, considérée comme la prédiction la plus probable, entourée de plusieurs zones de confiance aux teintes dégradées. Ces bandes visuelles illustrent l'incertitude du modèle face aux fluctuations futures du marché : plus l'intervalle est large, plus le réseau de neurones exprime un doute quant à la direction des prix.

Enfin, la dernière partie de ce bloc de code s'attache au formatage esthétique de la figure afin de la rendre exploitable pour une intégration dans un rapport académique. Le script attribue un titre explicite au graphique et utilise des fonctions de transformation géométrique pour extraire la mesure d'erreur de la prédiction (la perte mathématique) et la repositionner élégamment sous forme de légende sous l'axe des abscisses. Cette mise en page soignée permet d'obtenir une représentation finale à la fois rigoureuse scientifiquement et accessible visuellement, prête à être analysée pour valider ou infirmer nos hypothèses de recherche.

```

1 # for idx in range(10): # mise en graphique des 10 meilleures
   predictions
2     best_tft.plot_prediction(x, raw_predictions, idx=idx,
3                             add_loss_to_title=True,
4                             plot_attention=False)
5
6 import matplotlib.pyplot as plt
7
8 # On cree le "cadre" du graphique
9 fig, ax = plt.subplots(figsize=(10, 5))

```

```

10
11 # On dessine les predictions du modele par-dessus les vraies donnees
12 best_tft.plot_prediction(
13     x,
14     raw_predictions,
15     idx=0,
16     add_loss_to_title=True,
17     ax=ax
18 )
19
20 # On sauvegarde d'abord la loss generee automatiquement par le modele
21 texte_loss = ax.get_title()
22
23 ax.set_title("Prediction du cours de l'action Apple par le modele TFT")
24
25 # On place la loss sauvegardee en bas a gauche avec un petit fond
    lisible
26 ax.text(
27     0.02,
28     0.05,
29     texte_loss,
30     transform=ax.transAxes,
31     verticalalignment='bottom',
32     horizontalalignment='left',
33     bbox=dict(boxstyle='round', facecolor='white', alpha=0.8)
34 )
35
36 plt.show()

```

Génération des prédictions et analyse des performances par variable

Ce bloc de code permet d'auditer la précision de l'algorithme en analysant son taux d'erreur de manière isolée pour chaque variable explicative. La fonction `predict` génère d'abord les prévisions sur le jeu de validation tout en extrayant les données d'entrée grâce à l'argument `return_x=True`. Ensuite, la méthode `calculate_prediction_actual_by_variable` croise ces prédictions avec les observations réelles en les regroupant par catégorie temporelle. L'instruction finale `plot_prediction_actual_by_variable` traduit alors automatiquement ces calculs en une série de graphiques comparatifs..

```

1 predictions, x, *le_reste = best_tft.predict(
2     val_dataloader,
3     return_x=True
4 )
5
6 predictions_vs_actuals = best_tft.calculate_prediction_actual_by_
    variable(
7     x,
8     predictions
9 )
10
11 best_tft.plot_prediction_actual_by_variable(
12     predictions_vs_actuals
13 )

```

Analyse ciblée et représentation graphique sur un mois données (ici novembre)

Dans la stricte continuité de l'analyse précédente, cette section du code transpose notre méthodologie d'évaluation sur la deuxième période d'observation imposée par notre protocole de recherche, à savoir le mois de novembre. Le processus débute par l'application d'un filtre conditionnel rigoureux sur notre base de données d'apprentissage afin d'en extraire exclusivement les données historiques appartenant à ce marqueur temporel spécifique. Notre Temporal Fusion Transformer optimal est alors sollicité une première fois pour évaluer ce sous-ensemble et générer ses prévisions sous la forme de quantiles purs, fournissant ainsi une estimation numérique de la distribution des probabilités pour cette période de l'année.

Dans un second temps, le script réitère cette même opération de filtrage ciblé dans le but d'extraire conjointement les prédictions brutes du réseau de neurones et l'historique de la séquence. Cette extraction utilise de nouveau la syntaxe de dépaquetage sécurisée afin de garantir l'intégrité de l'exécution. Ces informations structurelles, liant le passé observé au futur estimé, sont finalement transmises à l'outil de traçage graphique intégré au modèle. Cette instruction ultime génère instantanément la représentation visuelle finale de ce mois de septembre, en superposant l'évolution réelle du marché aux bandes d'incertitude orangées calculées par l'intelligence artificielle. Cette démarche répétitive est essentielle pour notre rapport, car elle nous permet d'obtenir un point de comparaison direct avec le mois de novembre et d'étudier la constance des performances de notre algorithme face à des conjonctures temporelles distinctes.

```

1 nbm = max(data.cat_month_temporal_code)
2 eval1 = nbm - 1 # dernier mois de novembre
3 eval2 = nbm - 4 # dernier mois Aout
4 eval3 = nbm - 7 # dernier mois de mai
5
6 # Predictions de novembre (Quantiles)
7 best_tft.predict(
8     training.filter(lambda x: (x.cat_month_temporal_code == eval1)),
9     mode="quantiles",
10 )
11
12 # Representation graphique de novembre (Raw)
13 raw_prediction, x, *le_reste = best_tft.predict(
14     training.filter(lambda x: (x.cat_month_temporal_code == eval1)),
15     mode="raw",
16     return_x=True,
17 )
18
19 # On capture la figure generee par le modele
20 fig = best_tft.plot_prediction(
21     x,
22     raw_prediction,
23     idx=0,
24     add_loss_to_title=True,
25     plot_attention=False
26 )
27

```

```

28 # On recupere l'axe principal pour manipuler le texte
29 ax = fig.gca()
30
31 # On sauvegarde la loss generee par le modele
32 texte_loss = ax.get_title()
33
34 # On met un titre personnalise en haut
35 ax.set_title("Prediction AAPL du dernier mois de novembre")
36
37 # On ajoute la loss en bas a gauche avec un fond blanc
38 ax.text(
39     0.02,
40     0.05,
41     texte_loss,
42     transform=ax.transAxes,
43     verticalalignment='bottom',
44     bbox=dict(boxstyle='round', facecolor='white', alpha=0.8)
45 )
46
47 plt.show()

```

Interprétabilité du modèle et analyse de l'importance des variables

Cette toute dernière section du code illustre l'un des atouts technologiques majeurs du Temporal Fusion Transformer par rapport aux anciens réseaux de neurones classiques : sa remarquable capacité à expliquer ses propres décisions. Contrairement aux modèles de type "boîte noire" qui se contentent de délivrer un résultat final sans justification, le TFT intègre nativement des mécanismes permettant de comprendre précisément le cheminement mathématique qui a conduit à ses prédictions probabilistes. La première instruction commande ainsi à l'algorithme d'analyser ses propres résultats bruts globaux afin d'en extraire les métriques d'interprétabilité fondamentales. En appliquant une méthode de réduction par la somme, le programme agrège ces données complexes pour obtenir une vision macroscopique et représentative du comportement du modèle sur l'ensemble du jeu de données de validation.

L'aboutissement de cette démarche d'explicabilité se concrétise lors de l'exécution de la seconde instruction, qui a pour rôle de traduire ces calculs internes en graphiques lisibles. Ces représentations visuelles mettent en lumière deux axes d'analyse absolument cruciaux pour notre étude. D'une part, elles affichent l'importance relative de chaque variable sous forme de diagrammes, ce qui permet de vérifier instantanément si le modèle a accordé plus de poids aux volumes d'échanges, aux moyennes mobiles lissées, ou aux simples effets calendaires pour anticiper le marché. D'autre part, elles tracent la courbe de l'attention temporelle, révélant exactement sur quels jours du passé l'intelligence artificielle a focalisé son regard pour deviner l'avenir. L'intégration de ces graphiques dans notre rapport final est primordiale d'un point de vue scientifique, car elle permet de valider empiriquement que les prévisions de la machine reposent sur des signaux financiers cohérents et pertinents, et non sur des corrélations purement hasardeuses.

```
1 interpretation = best_tft.interpret_output(  
2     raw_predictions,  
3     reduction="sum"  
4 )  
5  
6 best_tft.plot_interpretation(interpretation)
```

5.2 Prédiction de la Bourse Apple cas Données normales vs bruitées AAPL

Dans cette section, le but est tout d'abord de montrer les différentes prédictions des 5 derniers jours des différentes bases de données grâce à l'historique des données sur une durée de 17 jours. Ensuite, nous établirons des comparaisons prédictives des derniers mois de Novembre, Août et Mai.

L'analyse comparative des prédictions sur la période de 8 ans (2011-2019 : T_8) met en évidence la réaction défensive immédiate du modèle face à la dégradation de l'information. Sur les données normales, l'algorithme génère une prédiction médiane stable autour de 10 avec un intervalle de confiance étroit et une perte limitée à 0.16. Pour y parvenir, il concentre toute son attention sur le tout premier jour de l'historique. À l'inverse, l'injection de bruit force le réseau de neurones à s'ajuster : si la prédiction centrale résiste en remontant très légèrement vers 10.5, l'incertitude explose. Les bandes de probabilité s'élargissent massivement de 9 à plus de 14 et la perte grimpe à 0.24. Désorienté par ce chaos, le modèle abandonne son point d'ancrage initial pour lisser son attention de manière diffuse et continue sur l'ensemble de la période.

Sur la période de 4 ans (2015-2019 : T_4), caractérisée par une échelle de prix beaucoup plus élevée, la confrontation entre les données saines et altérées confirme cette mécanique d'autodéfense avec une stratégie interne différente. Face au jeu sain, la prédiction se maintient près de la réalité, juste sous les 29, avec une incertitude modérée d'une amplitude de 5 unités et une attention répartie tout au long de la fenêtre temporelle. Face au bruit, l'algorithme devient plus pessimiste en sous-estimant le prix autour de 26 et double littéralement sa marge d'erreur pour atteindre une amplitude de 10 unités. De plus, sa gestion de l'attention s'inverse totalement par rapport à la période précédente : fuyant les signaux récents devenus illisibles, il s'ancre massivement sur la toute première observation de son historique avec un pic d'attention supérieur à 0.20, ignorant presque totalement le reste de la séquence.

Cependant, il est important de souligner un fait très visible. Le modèle adopte un comportement très conservateur. Face au chaos (données bruitées) et face à la série sur 4 années, la prédiction médiane (ligne orange) s'aplatit de manière excessive pour former une trajectoire presque horizontale. Ce qui pourrait être une sorte de sensibilité du TFT face à une taille faible de données ou une perturbation.

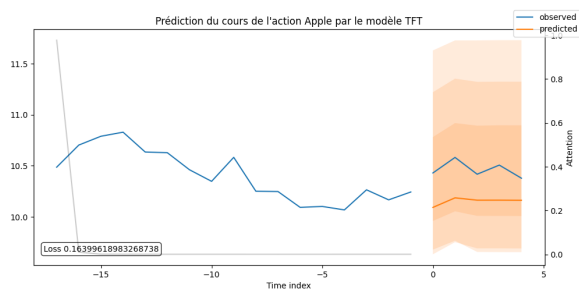


FIGURE 5 – Prédiction Bourse AAPL T_8 des données normales

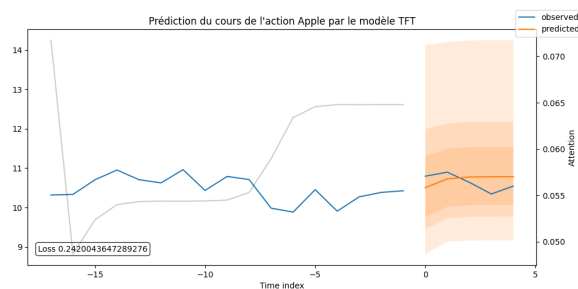


FIGURE 6 – Prédiction Bourse AAPL T_8 des données bruitées

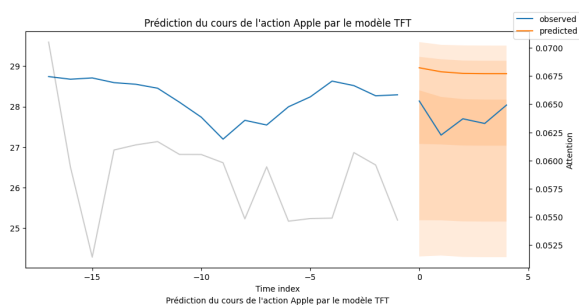


FIGURE 7 – Prédiction Bourse AAPL T_4 des données normales

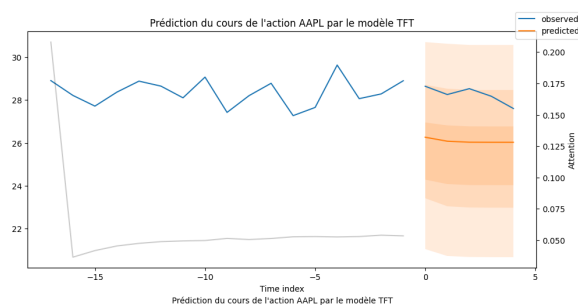


FIGURE 8 – Prédiction Bourse AAPL T_4 des données bruitées

Pour évaluer la qualité des prédictions sur un horizon mensuel, nous avons choisi d'isoler de manière arbitraire la période de mai. L'analyse basée sur un historique d'apprentissage de 8 ans (période T_8, allant de 2011 à 2019) confirme la bonne calibration du modèle. Sur les données saines, la perte par quantiles s'établit à un niveau satisfaisant de 0.32. Le modèle affiche une prédiction sereine avec une borne de sécurité restreinte à 6 dollars d'amplitude, comprise dans l'intervalle 41 à 47 dollars. L'injection du bruit dégrade logiquement cette perte qui monte à 0.65, forçant le modèle à élargir sa zone de couverture à 10 dollars d'amplitude (entre 38 et 48 dollars) pour compenser ce chaos. Le passage à un historique d'apprentissage réduit de moitié (période T_4, de 2015 à 2019) met en évidence une contrainte technique majeure de l'architecture TFT : sa forte dépendance au volume de données disponibles. En effet, même sur des données normales, la perte fait un bond spectaculaire à 0.69, soit plus du double du score obtenu avec 8 ans d'historique. Privé d'une profondeur temporelle suffisante, le modèle perd en assurance et ouvre d'emblée un intervalle de sécurité de 10 dollars d'amplitude (entre 36 et 46 dollars). Enfin, l'altération de cette courte série de 4 ans par le bruit gaussien pousse la perte à son niveau maximal de 0.73. Ce comportement démontre que si le modèle parvient à amortir l'incertitude grâce à son approche par quantiles, il atteint ses limites mathématiques et ne parvient plus à contrôler totalement le chaos lorsqu'il est simultanément privé d'informations historiques longues sur la bourse APPLE et est confronté à des signaux corrompus.

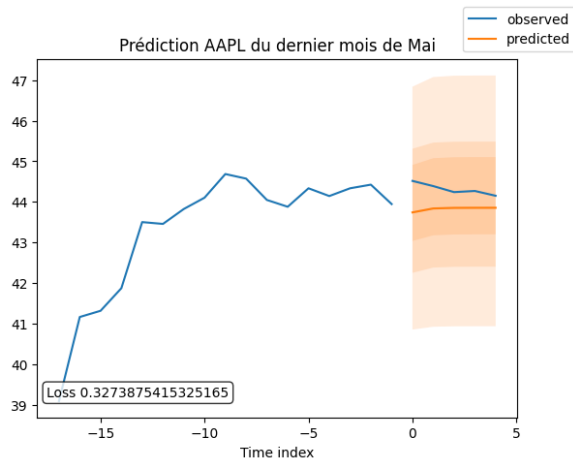


FIGURE 9 – Prédiction AAPL du mois de Mai T_8 – données normales

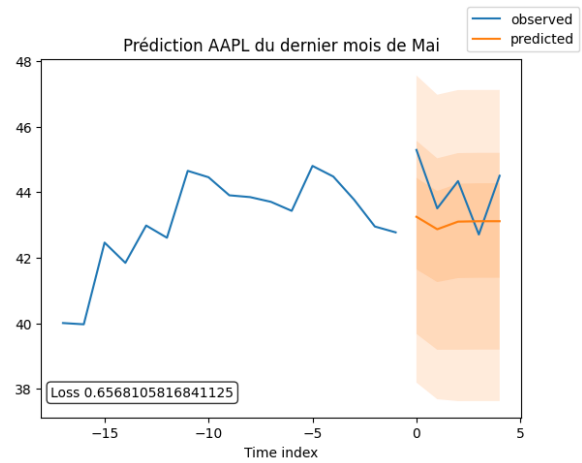


FIGURE 10 – Prédiction AAPL du mois de Mai T_8 – données bruitées

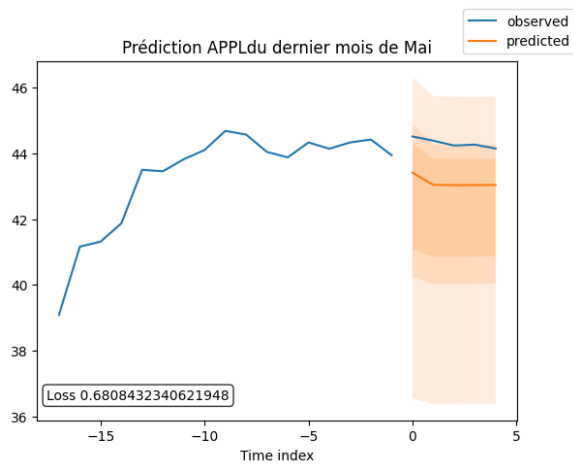


FIGURE 11 – Prédiction AAPL du mois de Mai T_4 – données normales

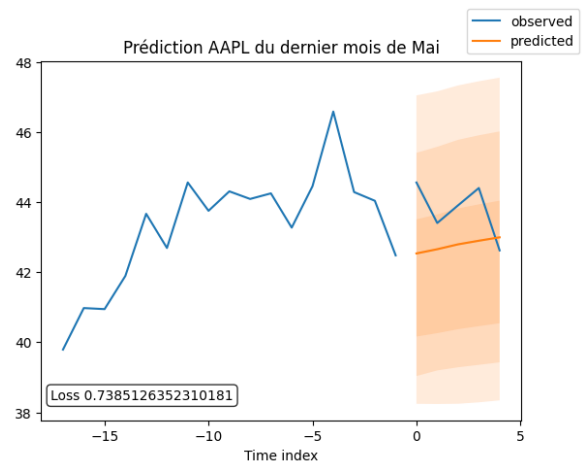


FIGURE 12 – Prédiction AAPL du mois de Mai T_4 – données bruitées

Les graphiques d'attention que nous observons ci-dessous sur les données d'une série de 8 ans et 4 ans représentent la moyenne macroscopique de l'importance accordée à chaque pas de temps passé pour chaque type et chaque série de données. Il s'agit du processus d'attention général d'attention de notre réseau de neurone sur tout le processus d'apprentissage pour les différentes séries.

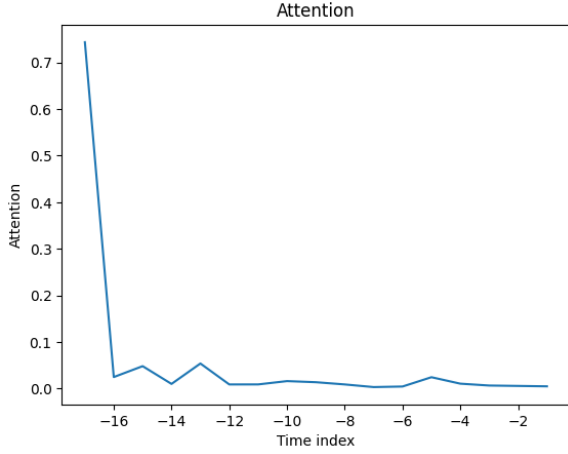


FIGURE 13 – Attention du modèle – données normales T_8ans

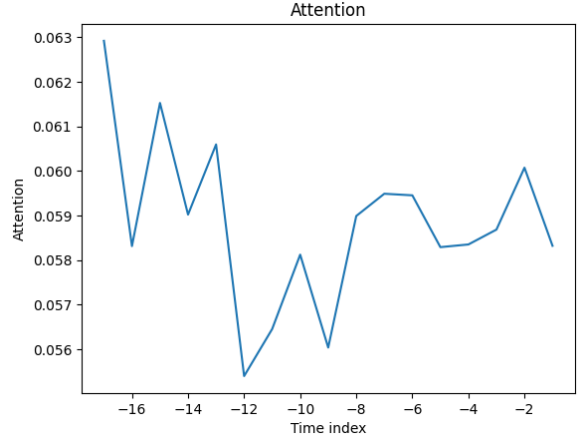


FIGURE 14 – Attention du modèle – données bruitées T_8ans

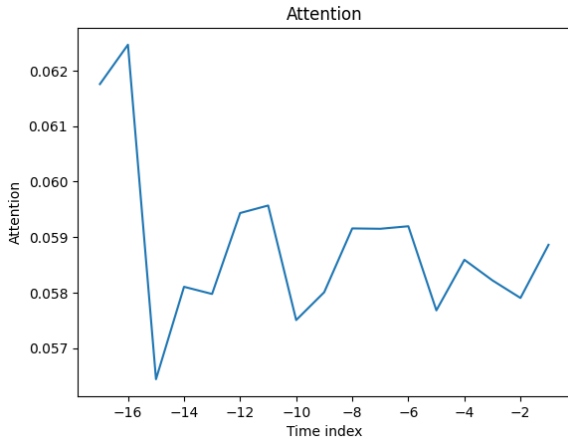


FIGURE 15 – Attention du modèle – données normales T_4ans

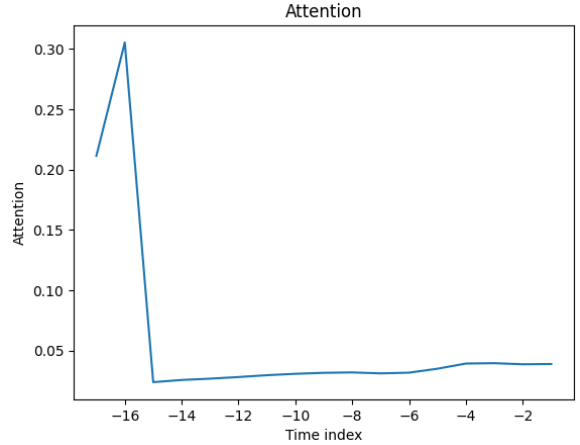


FIGURE 16 – Attention du modèle – données bruitées T_4ans

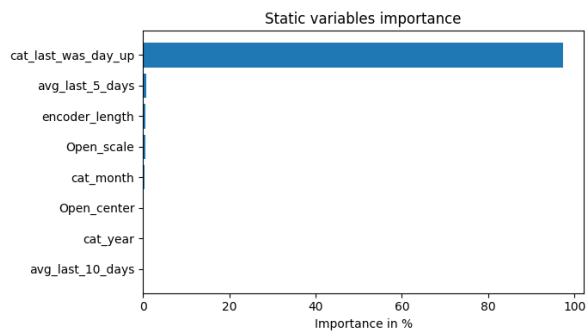
5.2.1 Réseaux de sélection de variables

Bien que plusieurs variables puissent être disponibles, leur pertinence et leur contribution spécifique au résultat sont généralement inconnues. Le TFT est conçu pour effectuer une sélection de variables pour chaque instance grâce à l'utilisation de réseaux de sélection de variables appliqués aux covariables statiques et dépendantes du temps. Outre l'identification des variables les plus significatives pour le problème de prédiction, la sélection de variables permet également à TFT d'éliminer les entrées parasites inutiles susceptibles d'affecter négativement les performances. La plupart des jeux de données de séries temporelles réelles contiennent des caractéristiques à faible pouvoir prédictif; la sélection de variables peut donc considérablement améliorer les performances du modèle en concentrant l'apprentissage sur les variables les plus pertinentes.

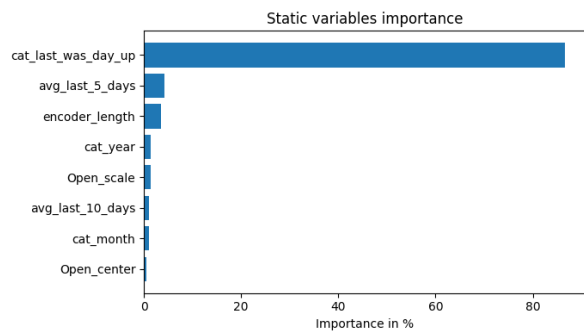
Dans notre étude, sur une série de 8 ans (2011-2019), la variable `cat_last_day_up` est la variable statique la plus importante pour les deux formes de données. Le scénario est complètement différent pour le cas des variables encodeurs. Les informations historiques que l'algorithme observe dans sa fenêtre d'apprentissage (nos 17 jours de passé) diffèrent

selon le type de données. Sur les données saines, la variable **volume** est la plus importante et est suivie par les variables **cat_month**, **Close**, et **High**, Alors que les variables **Open** et **Cat_year** sont les plus importantes pour encoder les données bruitées.

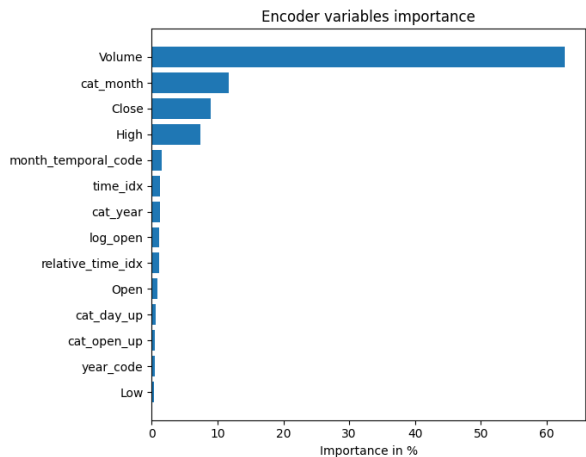
À l'inverse, les variables de décodeurs, ce sont les variables que nous connaissons avec une certitude absolue pour les jours à venir. Même si nous ignorons quel sera le prix de l'action Apple dans trois jours, nous savons en revanche très bien quel jour de la semaine nous serons, quel sera le mois en cours, ou s'il s'agira d'un jour férié (comme Noël). Le décodeur utilise ces variables temporelles futures pour construire une sorte de calendrier mathématique. S'agissant des normales, le TFT est plus focus sur les variables **mont_temporal_code**, **cat_year**, **time_idx**, un peu moins sur **cat_month** et **year_code** pour se prononcer sur l'avenir, alors que pour les données bruitées, pratiquement toute l'importance est portée sur **mont_temporal_code**, et d'une façon plus moindre sur **cat_year**. C'est scénario sont complètement différentes pour la série sur 4ans comme le montre les graphiques en annexe.



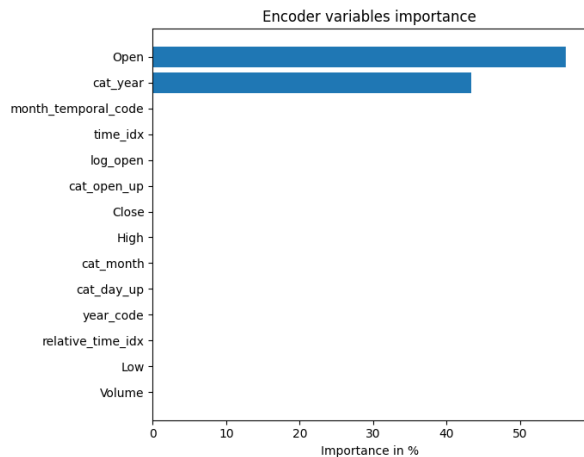
Variables statiques T8 – données normales



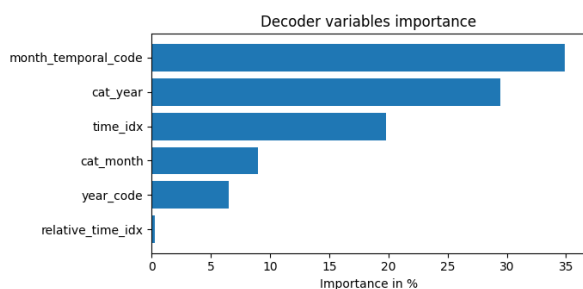
Variables statiques T8 – données bruitées



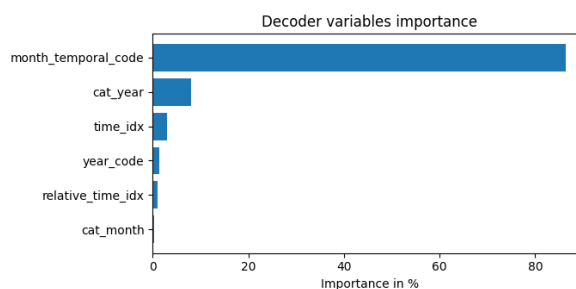
Encoder T8 – données normales



Encoder T8 – données bruitées



Decoder T8 – données normales



Decoder T8 – données bruitées

6 Limites du modèle TFT

6.1 Un comportement excessivement conservateur surtout face au chaos

La première limite empirique s'observe directement sur les graphiques confrontant le modèle aux données bruitées et les données saines d'une façon modérée. Face à une dégradation soudaine de l'information et à une incertitude extrême, le modèle adopte un mécanisme de défense très conservateur : la prédiction médiane (représentée par la ligne orange) s'aplatit de manière excessive pour former une trajectoire presque horizontale. En refusant de suivre des signaux devenus illisibles, l'algorithme privilégie une prudence statistique globale. Le risque inhérent à ce lissage est la perte de réactivité : dans un cas d'application réel, le modèle pourrait rater d'éventuels retournements brutaux ou des pics de marché.

6.2 Une perte d'exploitabilité financière liée à l'incertitude

La deuxième limite, corollaire de la première, concerne l'utilité opérationnelle de la prédiction pour un investisseur. Le TFT gère remarquablement le risque en élargissant ses bandes de probabilité, allant jusqu'à doubler son intervalle de confiance face au bruit (passant par exemple d'une amplitude de 5 dollars à plus de 10 dollars sur la période réduite T_4). Si cette ouverture de "parapluie" est une réussite mathématique prouvant que le modèle encadre bien la véritable valeur, elle devient un frein stratégique. Sur une action cotant autour de 25 ou 30 dollars, une fourchette d'incertitude de 10 dollars est beaucoup trop large pour déclencher un signal d'achat ou de vente précis et exploitable sur les marchés.

6.3 Une forte dépendance à la profondeur de l'historique d'apprentissage

La troisième limite confirmée par nos données concerne la sensibilité du modèle au volume d'informations disponibles. L'analyse ciblée sur les prédictions mensuelles (mois de mai) a clairement démontré qu'une réduction de la fenêtre d'apprentissage altérerait la confiance de l'algorithme. En passant d'un historique de 8 ans (série T_8) à un historique de 4 ans (série T_4) sur des données saines, la perte par quantiles a littéralement doublé, passant de 0.32 à 0.69. Le TFT a donc impérativement besoin d'une très grande profondeur chronologique pour calibrer correctement son réseau d'attention ; sans ce recul, ses

performances prédictives chutent drastiquement, même en l'absence de bruit artificiel.

7 Conclusion générale et perspective

Ce travail s'est attaché à explorer le potentiel du modèle Temporal Fusion Transformer (TFT) appliqué à la prédiction financière (notamment la bourse Apple sur la période 2011-2019), un domaine réputé pour sa volatilité extrême et son niveau de bruit structurel. À travers l'étude approfondie du cours de l'action Apple, nous avons pu démontrer que cette architecture marque une véritable rupture technologique par rapport aux réseaux de neurones récurrents traditionnels (tels que les LSTM). Là où les modèles classiques souffrent d'une dégradation chronologique du signal et agissent souvent comme des boîtes noires, le TFT s'est illustré par sa capacité à appréhender le temps de manière globale grâce à son mécanisme d'attention, tout en offrant une transparence inédite sur son processus de décision interne.

Nos multiples expériences de crash-tests, notamment par l'injection de bruit gaussien, ont mis en évidence la force majeure de cet algorithme : sa gestion probabiliste de l'incertitude. Face à un environnement devenu chaotique et illisible, le modèle ne se contente pas de fournir une estimation ponctuelle aveugle. Il adapte rationnellement sa stratégie en lissant son attention temporelle et en élargissant massivement ses intervalles de confiance pour encapsuler le risque. La fonction de perte par quantiles s'est ainsi révélée être un outil mathématique redoutable pour traduire l'imprévisibilité du marché en une mesure de risque visuelle et exploitable.

Néanmoins, la rigueur de cette étude a également permis d'identifier les limites empiriques de cette approche. Nos tests comparatifs sur des fenêtres d'apprentissage variables (de huit à quatre ans) ont prouvé la forte dépendance du modèle au volume et à la profondeur des données historiques. Privée d'un recul suffisant, l'architecture perd en assurance, ce qui se traduit par une multiplication par deux de sa perte mathématique. De surcroît, son mécanisme d'autodéfense face à une incertitude extrême induit un comportement très conservateur : la prédiction médiane a tendance à s'aplatir totalement, privilégiant la prudence au risque de rater d'éventuels retournements brutaux de marché.

Ces constats objectifs dessinent des perspectives de recherche futures très claires. Pour franchir un nouveau cap prédictif et s'affranchir de ce lissage de sécurité, la transition vers une modélisation fortement multivariée apparaît incontournable. Fournir au modèle des données macroéconomiques exogènes (évolution des taux directeurs, inflation) ou intégrer l'analyse de sentiment issue du traitement du langage naturel (NLP) sur la presse financière permettrait d'enrichir le contexte et de justifier des prises de position plus franches. De plus, l'application de cette architecture probabiliste à des classes d'actifs totalement décorrélées (matières premières, devises) constituerait un test décisif pour valider sa généralisation.

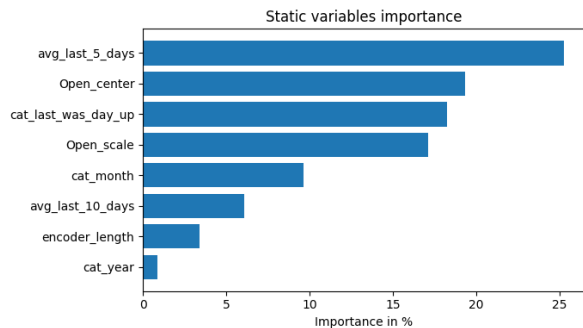
En définitive, si la prédiction parfaite des marchés financiers demeure une illusion mathématique, l'architecture TFT propose un changement de paradigme fondamental. Elle ne cherche plus seulement à deviner l'avenir, elle apprend à quantifier son propre doute, offrant ainsi une boussole analytique de premier plan pour l'aide à la décision financière.

Références

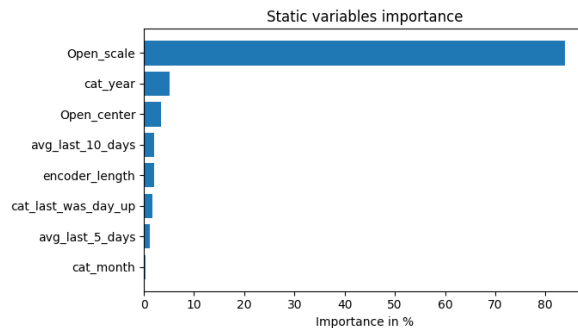
- Ahmed M. Alaa and Mihaela van der Schaar. Frequentist uncertainty in recurrent neural networks via blockwise influence functions. In *International Conference on Machine Learning (ICML)*, 2020.
- Emergent Mind. Temporal fusion transformer (tft). <https://www.emergentmind.com/topics/temporal-fusion-transformer-tft>, 2025. Accessed : 2026-03-06.
- Bryan Lim, Sercan O. Arik, Nicolas Loeff, and Tomas Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4) :1748–1764, 2021.
- Tristan Marcelin. Quantification d’incertitude d’un « long short- term memory » sur les marchés financiers. 2020.

Anexe

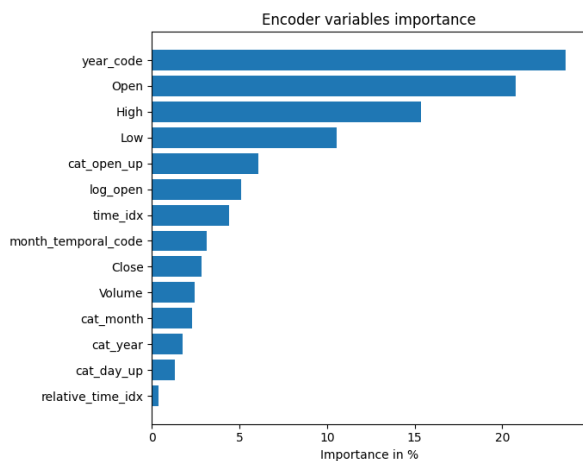
importance des Variables



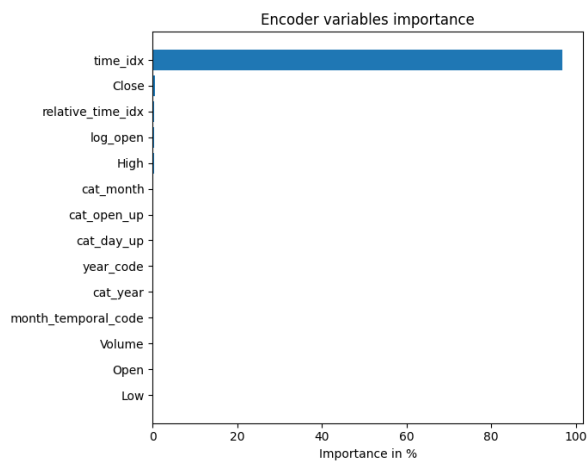
Variables statiques T4 – données normales



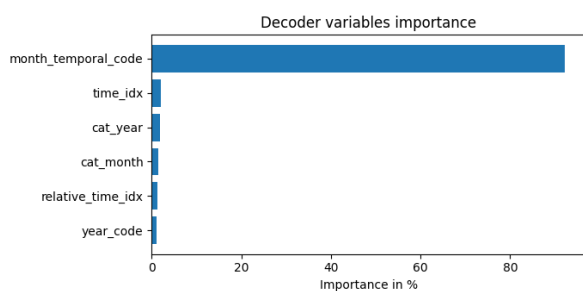
Variables statiques T4 – données bruitées



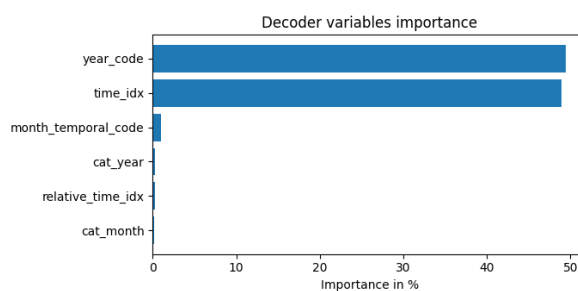
Encoder T4 – données normales



Encoder T4 – données bruitées

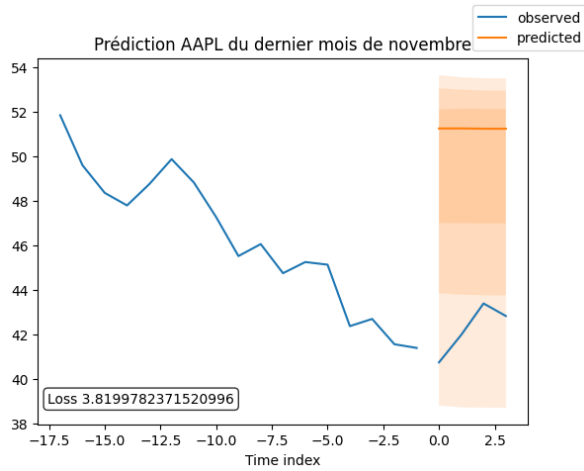


Decoder T4 – données normales

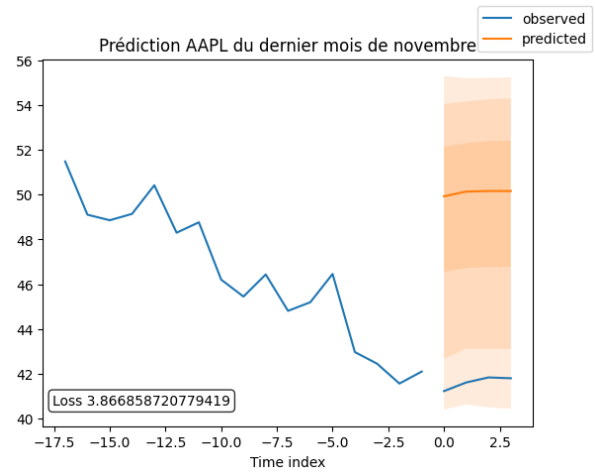


Decoder T4 – données bruitées

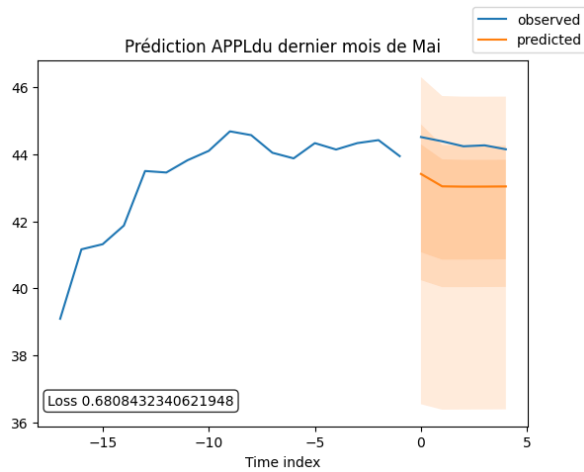
Prédiction sur quelques mois pour la période T_4



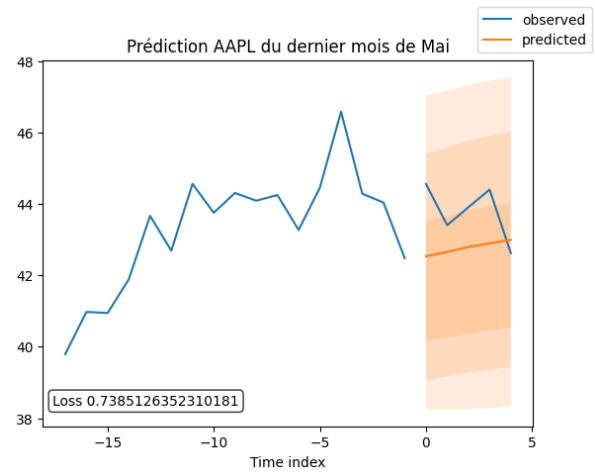
Prédiction novembre T4 – données normales



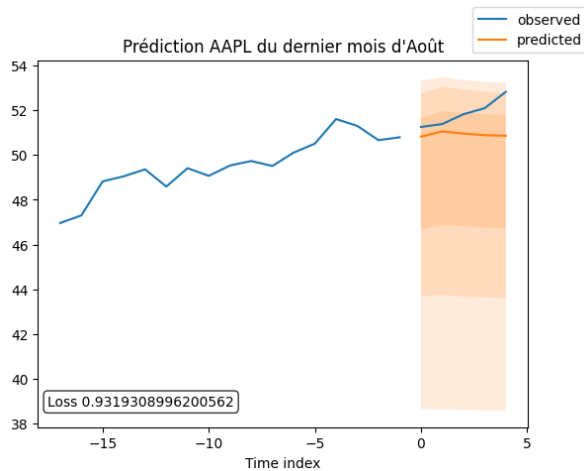
Prédiction novembre T4 – données bruitées



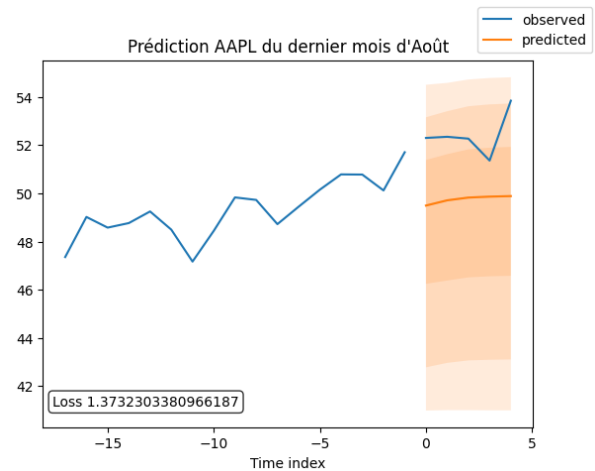
Prédiction mai T4 – données normales



Prédiction mai T4 – données bruitées

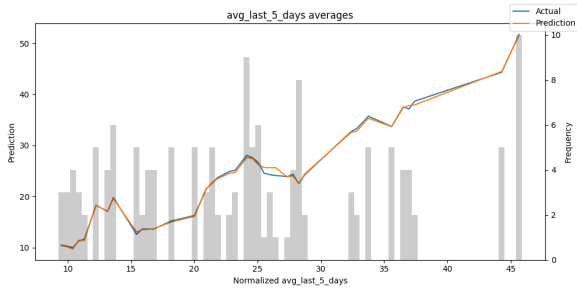


Prédiction août T4 – données normales

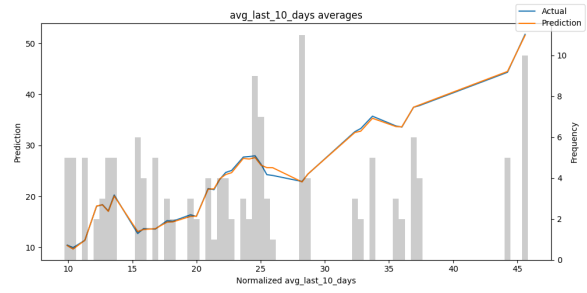


Prédiction août T4 – données bruitées

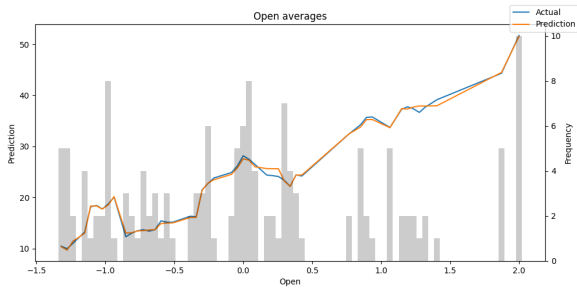
prédictions et analyse des performances par quelques variables



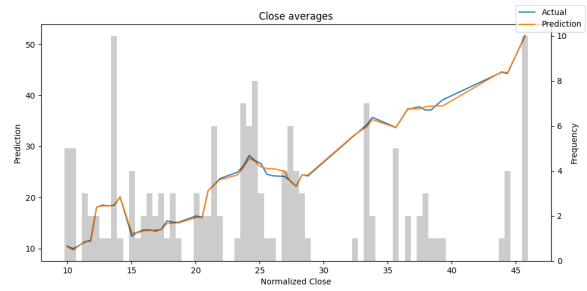
Données saines – avg_last_5_days



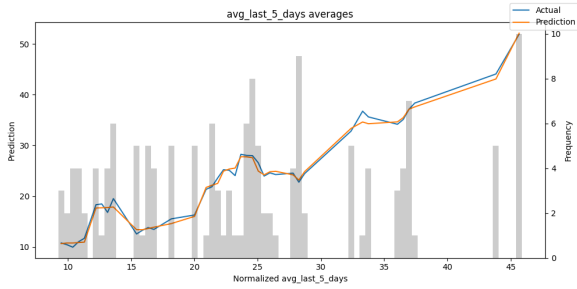
Données saines – avg_last_10_days



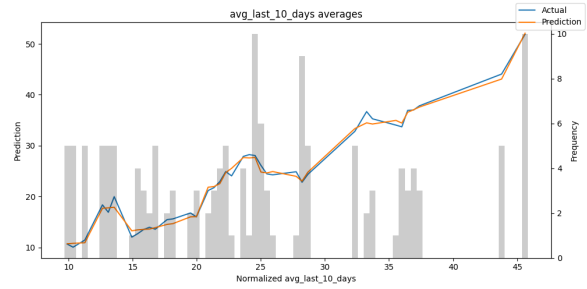
Données saines – open_avg



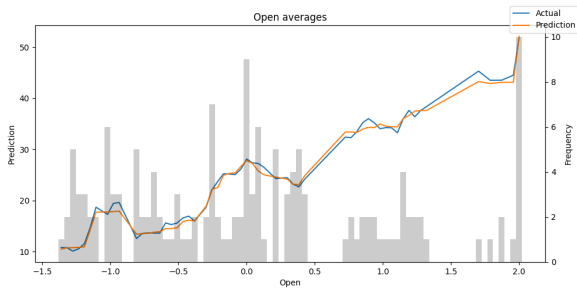
Données saines – close_avg



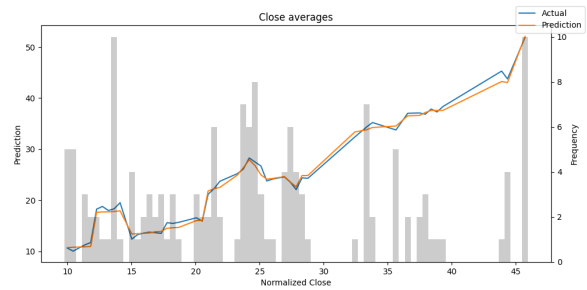
Données bruitées – avg_last_5_days



Données bruitées – avg_last_10_days



Données bruitées – open_avg



Données bruitées – close_avg